

LAB-01 DDL & DML Commands

* Structured Query Language (SQL):-

SQL which stands for Structured Query Language is a specialized programming language utilised for managing, quering and manipulating relational data bases

There are different types of SQL languages which are meant for special purpose. Some of them are

→DDL- Data definition language

→DML- Data Manipulation language

→DQL-Data Query language

→DCL- Data Control language.

→TCL- Transaction Control language.

→ DAL-Data Administration language.

Data Definition Language (DDL):

DDL is used for defining & managing the Structure of databases & their Components. DDL Statements include Commands like 'CREATE', 'ALTER', IDROP', these are used to create new databases, tabled, modify their Structures or even delete them.

DDL Commands:-

CREATE:-

The Create Command in DDL is used to create a Database Object Such as tables, views, triggers, indezes, etc

Creating a database: last want it to Create Commands Can be used to Create a database in following ways.

Syntax CREATE DATABASE DB-NAME

Example:- Let us Create a database for a university

```
CREATE DATABASE Univ;
```

```
USE Univ;  
SHOW DATABASES;
```

Creating a Table

we can create database instances such as using Create Command in following ways.

Syntax: CREATE TABLE Table Name (Attributes)

Example: Let us create a table of Students.

```
CREATE TABLE Student (SID VARCHAR(7), SNAME CHAR(1) NOT NULL, SEX CHAR (1) CHECK 1(SEX=  
'male' OR SEX = 'Female'), CGPA FLOAT);
```

ALTER:-

The 'Alter' Command in DDL is used to change or add Attributes of instances of Databases. Such as tables

Renaming a table! If we want to rename, ALTER Command along with RENAME Command used on the table.

Syntax: ALTER TABLE Table NAME RENAME to new name

Example: ALTER TABLE Student RENAME to newstudent; This way we can use Alter Command.

Similarly Renaming a Column of a table- Syntax: ALTER TABLE Table-name RENAME COLUMN Old-name to New-Name;

Example:

```
ALTER TABLE Student RENAME COLUMN SNAME TO STUDENT-NAME;
```

Adding a Column:-

If we want to add one or more columns to a table, we use 'ALTER' Command along with 'ADD' Command.

Syntax:-

```
ALTER TABLE Table-name ADD COLUMN Column-Name Datatype;
```

Example: Let us add a column for storing Address of existing Studens in database.

```
ALTER TABLE Student ADD Address VARCHAR(10);
```

Modifying a Column:-

If we want to modify datatype of a Column, we Can use 'ALTER' Command along with 'MODIFY' Command.

Syntax:

```
ALTER TABLE Table name MODIFY COLUMN NAME New datatype;
```

Example: Let us change the datatype of SEX from CHAR to VARCHAR.

```
ALTER TABLE Student MODIFY SEX VARCHAR(10);
```

DROP:-

DROP Command in DDL is used to drop columns in a table or Constraints of a column or even an entire database.

Let us see syntaxes for each with respective examples

Syntax:-

```
DROP DATABASE DB-name;
```

Example:- Let us delete Univ database

```
DROP DATABASE UNIV;
```

→ It is going to delete the entire database.

DROP a Table:-

We Can drop a table using 'DROP' Command

Syntax:-

```
DROP TABLE Table-name';
```

Example:- Let us delete Student table.

DROP TABLE Student;

DROPING a Column of a table:-

we can drop a Column of a table using 'DROP' Command

Syntax:-

ALTER TABLE Table-name DROP COLUMN Column-Name;

Example:- Let us delete SEX Column

ALTER TABLE Student DROP COLUMN SEX;

DATA MANIPULATING LANGUAGE

Data manipulating language (DML) is used to manipulate update or change the State of instance of database.

1) Three main command in DML:

1. INSERT
2. DELETE
3. UPDATE

1. **INSERT:** "INSERT" Command is used to insert data into specific columns or entire table.

SYNTAX : INSERT INTO TABLE_NAME(col., col2) VALUES (value1, Value2)

Example: let us insert same data into student table.

- 2) INSERT INTO STUDENT VALUES("N201076","JAGADEESH", 49),
("N008993", "girsh", 20),
("N201023","pradeep", 19);
- 3) SELECT * FROM STUDENT;

```
mysql> select * from STUDENT;
+-----+-----+-----+
| ID      | NAME      | AGE  |
+-----+-----+-----+
| N201076 | JAGADEESH | 49   |
| N008993 | girsh     | 20   |
| N201023 | pradeep   | 19   |
+-----+-----+-----+
3 rows in set (0.00 sec)
```

2. **DELETE:** "DELETE" command is used to delete. instances of a table or records of a table where a certain condition is met as we can delete entire table instances at once.

- 4) **Deleting every instance**

➤ DELETE From table-name;

5) Deleting rows based on condition

Syntax: DELETE from table-name WHERE condition;

Example: Let us delete records of students who got less than 20.

6) DELETE FROM STUDENT WHERE AGE<20;

7) SELECT * FROM STUDENT;

```
mysql> SELECT * FROM STUDENT;
+-----+-----+-----+
| ID      | NAME      | AGE  |
+-----+-----+-----+
| N201076 | JAGADEESH | 49   |
| N008993 | girsh     | 20   |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
```

3. **UPDATE:** update command is used to Change update some parts of data in a table

i) changing the values in column where in columns where condition is satisfied.

Syntax: UPDATE TABLE_NAME SET COL1 = new vol1, col2 = new VOL2 where condition;

Example: Let us update the age is 26 where Student ID is "N201076";

8) UPDATE STUDENT SET age=26 WHERE ID= "N201076";

9) SELECT * FROM STUDENT;

```
mysql> SELECT * FROM STUDENT;
+-----+-----+-----+
| ID      | NAME      | AGE  |
+-----+-----+-----+
| N201076 | JAGADEESH | 49   |
| N008993 | girsh     | 20   |
+-----+-----+-----+
2 rows in set (0.00 sec)

mysql>
```

LAB-02 BASIC SQL QUERIES

Aim:- To study the basic SQL Queries

Basic SQL Queries are:

- | | |
|-------------|-------------|
| 4. SELECT | 5. ORDER BY |
| 5. WHERE | 6. LIKE |
| 6. GROUP BY | 7. DESCRIBE |
| 7. HAVING | 8. COUNT |

#CREATION OF DATABASE

```
Microsoft Windows [Version 10.0.19045.3693]
(c) Microsoft Corporation. All rights reserved.

C:\Users\lenovo>cd C:\Program Files\MySQL\MySQL Workbench 8.0

C:\Program Files\MySQL\MySQL Workbench 8.0>mysql -u root -p
Enter password: *****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.34 MySQL Community Server - GPL

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> create database sample;
Query OK, 1 row affected (0.24 sec)

mysql> use sample;
Database changed
```

#CREATING STUDENT TABLE AND INSERTING VALUES IN IT

```
mysql> create table student(id int primary key,name varchar(20),sex varchar(10),gpa float);
Query OK, 0 rows affected (0.21 sec)

mysql> insert into student values(1,'akhila','female',9.8),
-> (2,'sravani','female',9.8),
-> (3,'bhavani','female',9.5),
-> (4,'asritha','female',10),
-> (5,'sravya','female',9.8),
-> (6,'gayathri','female',9.5);
Query OK, 6 rows affected (0.05 sec)
Records: 6  Duplicates: 0  Warnings: 0
```

SELECT:-

The **SELECT** statement is used to select data from a database table which returns this data in the form of a table.

One can fetch either the entire table or according to some specified rules.

Syntax:-

SELECT column1, column2, ...

FROM table_name;

-selects particular columns from the table

SELECT * from table_name; -selects all columns from the table.

Example:-

-Retrieving all the details from the student table

Query:-

>SELECT * FROM student;

Output:-

```
mysql> select * from student;
+----+-----+-----+-----+
| id | name   | sex   | gpa  |
+----+-----+-----+-----+
| 1  | akhila | female | 9.8  |
| 2  | sravani | female | 9.8  |
| 3  | bhavani | female | 9.5  |
| 4  | asritha | female | 10   |
| 5  | sravya  | female | 9.8  |
| 6  | gayathri | female | 9.5  |
+----+-----+-----+-----+
6 rows in set (0.00 sec)
```

Example:-

-Retrieving the names of the students

Query:-

>SELECT name FROM student;

OUTPUT:-

```
mysql> select name from student;
+-----+
| name   |
+-----+
| akhila |
| sravani |
| bhavani |
| asritha |
| sravya  |
| gayathri |
+-----+
6 rows in set (0.00 sec)
```

WHERE:-

The WHERE clause is used to filter the records.

It is used to extract only those records that fulfill a specified condition.

Syntax:-

SELECT column1,column2 FROM table_name WHERE [condition];

Example:-

Retrieving the student details whose gpa is greater than 9.5

Query:-

>SELECT * FROM student where gpa>9.5;

Output:-

```
mysql> select * from student where gpa>9.5;
+----+-----+-----+-----+
| id | name   | sex   | gpa  |
+----+-----+-----+-----+
| 1  | akhila | female | 9.8  |
| 2  | sravani | female | 9.8  |
| 4  | asritha | female | 10   |
| 5  | sravya | female | 9.8  |
+----+-----+-----+-----+
4 rows in set (0.00 sec)
```

GROUP BY:-

The GROUP BY clause causes the rows of the items table to be collected into groups, each group composed of rows that have identical order_num values

Syntax:

>SELECT column_name(s) FROM table_name GROUP BY column_name(s);

Example:-

-Retrieving the details of students group by gpa

Query:-

>SELECT count(*),gpa FROM student group by gpa;

Output:-

```
mysql> select count(*),gpa from student group by gpa;
+-----+-----+
| count(*) | gpa  |
+-----+-----+
| 3        | 9.8  |
| 2        | 9.5  |
| 1        | 10   |
+-----+-----+
3 rows in set (0.00 sec)
```

HAVING:-

The HAVING clause is used to allow the filtering of query results based on aggregate functions and groupings, which cannot be achieved using the WHERE clause that is used to filter individual rows.

- HAVING clause is only used with the SELECT clause
- HAVING clause is generally used after GROUP BY

Syntax:-

HAVING <group qualification condition(s)>;

Example:-

-Retrieving the names of students whose gpa lies between 9 and 10 and group them by gpa where such group contain at least 2 students.

Query:-

```
> SELECT gpa , count(*) FROM student WHERE gpa between 9 and 10 GROUP BY gpa HAVING count(*)>=2;
```

Output:-

```
mysql> select gpa,count(*) from student where gpa between 9 and 10 group by gpa having count(*)>=2;
+-----+-----+
| gpa | count(*) |
+-----+-----+
| 9.8 | 3 |
| 9.5 | 2 |
+-----+-----+
2 rows in set (0.02 sec)
```

ORDER BY:-

The SQL ORDER BY is used to sort the data in either ascending or descending order,based on one or more columns.

The ORDER BY keyword sorts the records in ascending order by default.To sort the record in descending order,use the DESC keyword.

Syntax:-

```
SELECT column1,column2,.....column n
```

```
FROM table_name
```

```
ORDER BY column_name(s) ASC|DESC;
```

Example:-

Retrieving the student details in decreasing order of their gpa

Query:-

```
>SELECT * from Student order by gpa DESC;
```

```
mysql> Select * from student order by gpa desc;
+----+-----+-----+-----+
| id | name   | sex   | gpa |
+----+-----+-----+-----+
| 4  | asritha | female | 10  |
| 1  | akhila  | female | 9.8 |
| 2  | sravani | female | 9.8 |
| 5  | sravya  | female | 9.8 |
| 3  | bhavani | female | 9.5 |
| 6  | gayathri | female | 9.5 |
+----+-----+-----+-----+
```

Output:- 6 rows in set (0.00 sec)

LIKE:-

The LIKE operator is used in a WHERE clause to search for a specified pattern in a column. There two wildcards often used in conjunction with the LIKE operator:

- The percent sign '%' represents zero, one, or multiple characters.
- The underscore sign '_' represents one, single character.

Syntax:-

WHERE column LIKE pattern;

Example:-

-Retrieving the details of female students whose name contains only 6 characters.

Query:-

>SELECT * FROM student where name LIKE '_____' and sex='female';

Output:-

```
mysql> select * from student where name LIKE '_____' and sex='female';
+----+-----+-----+-----+
| id | name   | sex   | gpa   |
+----+-----+-----+-----+
| 1  | akhila | female | 9.8   |
| 5  | sravya | female | 9.8   |
+----+-----+-----+-----+
2 rows in set (0.00 sec)
```

2)Retrieving the details of students whose name starts with character 'a'

Query:-

≥ SELECT * FROM student where name LIKE 'a%';

Output:-

```
mysql> select * from student where name LIKE 'a%';
+----+-----+-----+-----+
| id | name   | sex   | gpa   |
+----+-----+-----+-----+
| 1  | akhila | female | 9.8   |
| 4  | asritha | female | 10    |
+----+-----+-----+-----+
2 rows in set (0.00 sec)
```

DESCRIBE:-

DESCRIBE is used to describe something,in a table DESCRIBE or DESC commands are used to describe the structure of a table.

Syntax:-

DESC table_name; or DESCRIBE table_name;

Example:-

Description of table

Query:-

>DESC student;

Output:-

```
mysql> Desc student;
+-----+-----+-----+-----+-----+-----+
| Field | Type          | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| id    | int           | NO   | PRI | NULL    |       |
| name  | varchar(20)   | YES  |     | NULL    |       |
| sex   | varchar(10)   | YES  |     | NULL    |       |
| gpa   | float         | YES  |     | NULL    |       |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.01 sec)
```

COUNT:-

The count() function returns the number of rows that matches a specified criteria.

Null values are not counted and is used with the SQL SELECT statement.

Syntax:-

SELECT COUNT(expression) FROM table-name;

Example:-

-Retrieving the number of students

Query:-

>SELECT count(id) FROM student;

Output:-

```
mysql> SELECT count(id) FROM student;
+-----+
| count(id) |
+-----+
|          6 |
+-----+
1 row in set (0.03 sec)
```

LAB-03 STRING FUNCTIONS

1. CHAR_LENGTH()

Description: Returns the number of characters in a string.

Syntax: CHAR_LENGTH(string)

Example:

```
SELECT CHAR_LENGTH('hello') as Length;
```

```
+-----+
| Length |
+-----+
|    5   |
+-----+
```

2. CONCAT()

Description: Concatenates two or more strings.

Syntax: CONCAT(string1, string2, ...)

Example:

```
SELECT CONCAT('Good ', 'Morning!') as Greetings;
```

```
+-----+
| Greetings |
+-----+
| Good Morning! |
+-----+
```

3. CONCAT_WS()

Description: Concatenates two or more strings with a specified separator.

Syntax: CONCAT_WS(separator, string1, string2, ...)

Example:

```
SELECT CONCAT_WS('@', 'Good ', 'Morning!') as Greetings;
```

```
+-----+
| Greetings |
+-----+
| Good @Morning! |
+-----+
```

4. FIND_IN_SET()

Description: Searches for a value within a comma-separated list of values.

Syntax: FIND_IN_SET(value, comma-separated-list)

Example:

```
SELECT FIND_IN_SET(",", "s,,q,l") as Position;
```

```
+-----+
| Position |
+-----+
|    0    |
+-----+
```

5. FIELD()

Description: Returns the index (position) of the first occurrence of a value in a list of values. Syntax: FIELD(value, value1, value2, ...)

Example:

```
SELECT FIELD("q", "s", "q", "l") as Position;
```

```
+-----+
| Position |
+-----+
|    2    |
+-----+
```

6. FORMAT()

Description: Formats a number with a specified number of decimal places.

Syntax: FORMAT(number, decimal-places)

Example:

```
SELECT FORMAT(284943.983498, 2) as FormattedNumber;
```

```
+-----+
| FormattedNumber |
+-----+
| 284,943.98      |
+-----+
```

7. INSERT()

Description: Inserts a substring into a string at a specified position.

Syntax: INSERT(string, position, length, substring)

Example:

```
SELECT INSERT("mysqldbms.com", 1, 9, "Example") as Result;
```

```
+-----+
| Result      |
+-----+
| Example.com |
+-----+
```

8. INSTR()

Description: Returns the position of the first occurrence of a substring in a string.

Syntax: INSTR(string, substring)

Example:

```
SELECT INSTR("mysqldbms.com", "d") as MatchPosition;
```

```
+-----+
| MatchPosition |
+-----+
|          6    |
+-----+
```

9. LCASE()

Description: Converts a string to lowercase.

Syntax: LCASE(string)

Example:

```
SELECT LCASE("HELLO WORLD!") as LowercaseString;
```

```
+-----+
| LowercaseString |
+-----+
| hello world!    |
+-----+
```

10. LEFT()

Description: Extracts a specified number of characters from the beginning of a string.

Syntax: LEFT(string, number-of-characters)

Example:

```
SELECT LEFT('Hello world', 5) as ExtractedString;
```

```
+-----+
| ExtractedString |
+-----+
| Hello           |
+-----+
```

11. RIGHT()

Description: Extracts a specified number of characters from the end of a string.

Syntax: RIGHT(string, number-of-characters)

Example:

```
SELECT RIGHT('Hello world', 5) as ExtractedString;
```

```
+-----+
```

ExtractedString
world

12. LENGTH()

Description: Returns the length (number of characters) of a string.

Syntax: LENGTH(string)

Example:

SELECT LENGTH("Hello world") as LengthOfString;

LengthOfString
11

13. LOCATE()

Description: Returns the position of the first occurrence of a substring in a string.

Syntax: LOCATE(substring, string)

Example:

SELECT LOCATE("s", "Welcome to MySQL") as MatchPosition;

MatchPosition
14

14. LPAD()

Description: Pads a string to a specified length with another string.

Syntax: LPAD(string, length, pad-string)

Example:

SELECT LPAD("SQL Tutorial", 20, "ABC") as PaddedString;

PaddedString
ABCABCABSQSQL Tutorial

15. LOWER()

Description: Converts a string to lowercase.

Syntax: LOWER(string)

Example:

SELECT LOWER("Hello world") as LowercaseString;

```
+-----+
| LowercaseString |
+-----+
| hello world     |
+-----+
```

16. UPPER()

Description: Converts a string to uppercase.

Syntax: UPPER(string)

Example:

SELECT UPPER("Hello world") as UppercaseString;

```
+-----+
| UppercaseString |
+-----+
| HELLO WORLD     |
+-----+
```

17. LTRIM()

Description: Removes leading spaces from a string.

Syntax: LTRIM(string)

Example:

SELECT LTRIM(" Hello world!") as TrimmedString;

```
+-----+
| TrimmedString   |
+-----+
| Hello world!    |
+-----+
```

18. RTRIM()

Description: Removes trailing spaces from a string.

Syntax: RTRIM(string)

Example:

SELECT RTRIM("Hello!!!!!! ") as TrimmedString;

```
+-----+
| TrimmedString   |
+-----+
```


Hello!!!!!!

19. TRIM()

Description: Removes leading and trailing spaces from a string.

Syntax: TRIM(string)

Example:

SELECT TRIM(" Hello!!!!!! ") as TrimmedString;

TrimmedString
Hello!!!!!!

20. MID()

Description: Extracts a substring from a string starting at a specified position.

Syntax: MID(string, start-position, length)

Example:

SELECT MID("SQL Tutorial", 5, 5) as ExtractedString;

ExtractedString
Tutor

21. POSITION()

Description: Returns the position of a substring in a string.

Syntax: POSITION(substring IN string)

Example:

SELECT POSITION("!" IN "Hello world!") as Position;

Position
12

NUMERICAL FUNCTIONS

1. ABS()

Description: Returns the absolute (positive) value of a number.

Syntax: ABS(number)

Example:

```
SELECT ABS(-243.19) as AbsoluteValue;
```

+-----+	
AbsoluteValue	
+-----+	
243.19	
+-----+	

2. COS()

Description: Returns the cosine of an angle (in radians).

Syntax: COS(angle)

Example:

```
SELECT COS(1.57) as CosineValue;
```

+-----+	
CosineValue	
+-----+	
0.0007963267107332633	
+-----+	

3. ASIN()

Description: Returns the arcsine of a number.

Syntax: ASIN(number)

Example:

```
SELECT ASIN(0.5) as ArcsineValue;
```

+-----+	
ArcsineValue	
+-----+	
0.5235987755982989	
+-----+	

4. LOG()

Description: Returns the natural logarithm of a number.

Syntax: LOG(number)

Example:

```
SELECT LOG(2.7199) as NaturalLog;
```

+-----+	
NaturalLog	
+-----+	
1.0005951149261851	
+-----+	

5. PI()

Description: Returns the mathematical constant Pi (π).

Syntax: PI()

Example:

```
SELECT PI() as PiValue;
```

```
+-----+
| PiValue |
+-----+
| 3.141593 |
+-----+
```

6. MOD()

Description: Returns the remainder of the division of two numbers.

Syntax: MOD(dividend, divisor)

Example:

```
SELECT MOD(183, 4) as Remainder;
```

```
+-----+
| Remainder |
+-----+
|      3      |
+-----+
```

7. POWER()

Description: Raises a number to a specified power.

Syntax: POWER(base, exponent)

Example:

```
SELECT POWER(4, 2) as Result;
```

```
+-----+
| Result |
+-----+
|    16    |
+-----+
```

8. RADIANS()

Description: Converts degrees to radians.

Syntax: RADIANS(degrees)

Example:

```
SELECT RADIANS(180) as RadiansValue;
```

```
+-----+
| RadiansValue |
+-----+
```

```
| 3.141592653589793 |
```

```
+-----+
```

9. RAND()

Description: Generates a random floating-point number between 0 and 1.

Syntax: RAND()

Example:

```
SELECT RAND() as RandomNumber;
```

```
+-----+
```

```
| RandomNumber      |
```

```
+-----+
```

```
| 0.3438212483598117 |
```

```
+-----+
```

10. RAND()

Description: Generates a random floating-point number with a specified seed.

Syntax: RAND(seed)

Example:

```
SELECT RAND(7) as RandomNumberWithSeed;
```

```
+-----+
```

```
| RandomNumberWithSeed |
```

```
+-----+
```

```
| 0.9065021936842261  |
```

```
+-----+
```

11. ROUND()

Description: Rounds a number to a specified number of decimal places.

Syntax: ROUND(number, decimal-places)

Example:

```
SELECT ROUND(32124.98521, 2) as RoundedNumber;
```

```
+-----+
```

```
| RoundedNumber |
```

```
+-----+
```

```
| 32124.99      |
```

```
+-----+
```

12. SQRT()

Description: Returns the square root of a number.

Syntax: SQRT(number)

Example:

```
SELECT SQRT(155) as SquareRoot;
```

SquareRoot
12.449899597988733

13. SIGN()

Description: Returns the sign of a number (1 for positive, -1 for negative, 0 for zero).

Syntax: SIGN(number)

Example:

SELECT SIGN(-123) as Sign;

Sign
-1

14. TRUNCATE()

Description: Truncates a number to a specified number of decimal places.

Syntax: TRUNCATE(number, decimal-places)

Example:

SELECT TRUNCATE(92890.29802, 4) as TruncatedNumber;

TruncatedNumber
92890.2980

15. GREATEST()

Description: Returns the greatest (maximum) value among a list of values.

Syntax: GREATEST(value1, value2, ...)

Example:

SELECT GREATEST(12, 34, 9, 134, 74) as GreatestValue;

GreatestValue
134

LAB-04 DATE TIME FUNCTIONS

1. ADDDATE():

- **Description:** Adds a specified interval to a date.
- **Syntax:** ADDDATE(date, interval)
- **Example:** SELECT ADDDATE('2023-10-15', INTERVAL 2 YEAR);
- **Output:**

```
+-----+
| ADDDATE('2023-09-15', INTERVAL 2 YEAR) |
+-----+
| 2025-10-15                               |
+-----+
```

2. ADDTIME():

- **Description:** Adds a specified time to a datetime.
- **Syntax:** ADDTIME(datetime, time)
- **Example:** SELECT ADDTIME('2023-06-15 09:34:21', '2:10:05.00001');
- **Output:**

```
+-----+
| ADDTIME('2023-06-15 09:34:21','2:10:05.00001') |
+-----+
| 2023-06-15 11:44:26.000010                     |
+-----+
```

3. CURDATE():

- **Description:** Returns the current date.
- **Syntax:** CURDATE()
- **Example:** SELECT CURDATE();
- **Output:**

```
+-----+
| CURDATE()   |
+-----+
| 2023-12-14   |
+-----+
```

4. SYSDATE():

- **Description:** Returns the current date and time.
- **Syntax:** SYSDATE()
- **Example:** SELECT SYSDATE();
- **Output:**

```
+-----+
| SYSDATE()   |
+-----+
| 2023-12-14 23:10:28 |
+-----+
```

5. CURRENT_DATE():

- **Description:** Returns the current date.
- **Syntax:** CURRENT_DATE()
- **Example:** SELECT CURRENT_DATE();
- **Output:**

CURRENT_DATE()
2023-12-14

6. CURTIME():

- **Description:** Returns the current time.
- **Syntax:** CURTIME()
- **Example:** SELECT CURTIME();
- **Output:**

CURTIME()
23:13:51

7. CURRENT_TIMESTAMP():

- **Description:** Returns the current date and time.
- **Syntax:** CURRENT_TIMESTAMP()
- **Example:** SELECT CURRENT_TIMESTAMP()+100;

- **Output:**

CURRENT_TIMESTAMP()+100
20231214231623

8. DAY():

- **Description:** Returns the day of the month for a given date.
- **Syntax:** DAY(date)
- **Example:** SELECT DAY('2023-08-25');
- **Output:**

DAY('2023-08-25')
25

9. DATE_FORMAT():

- **Description:** Formats a date as specified.
- **Syntax:** DATE_FORMAT(date, format)
- **Example:** SELECT DATE_FORMAT(CURDATE(), '%Y %D %j %w');
- **Output:**

```
+-----+
| DATE_FORMAT(CURDATE(), '%Y %D %j %w') |
+-----+
| 2023 14th 348 4 |
+-----+
```

10. DATE_SUB():

- **Description:** Subtracts a specified interval from a date.
- **Syntax:** DATE_SUB(date, interval)
- **Example:** SELECT DATE_SUB('2023-09-15', INTERVAL 10 DAY);
- **Output:**

```
+-----+
| DATE_SUB('2023-09-15', INTERVAL 10 DAY) |
+-----+
| 2023-09-05 |
+-----+
```

11. DAYNAME():

- **Description:** Returns the name of the day for a given date.
- **Syntax:** DAYNAME(date)
- **Example:** SELECT DAYNAME('2023-10-15');
- **Output:**

```
+-----+
| DAYNAME('2023-10-15') |
+-----+
| Sunday |
+-----+
```

12. DAYOFWEEK():

- **Description:** Returns the day of the week for a given date.
- **Syntax:** DAYOFWEEK(date)
- **Example:** SELECT DAYOFWEEK('2023-10-15');
- **Output:**

```
+-----+
| DAYOFWEEK('2023-10-15') |
+-----+
| 1 |
+-----+
```


13. MINUTE():

- **Description:** Returns the minute for a given time or datetime.
- **Syntax:** MINUTE(time)
- **Example:** SELECT MINUTE('2023-09-15');
- **Output:**

```
+-----+
| MINUTE('2023-09-15') |
+-----+
|           20          |
+-----+
```

14. HOUR():

- **Description:** Returns the hour for a given time or datetime.
- **Syntax:** HOUR(time)
- **Example:** SELECT HOUR('2023-09-15');
- **Output:**

```
+-----+
| HOUR('2023-09-15') |
+-----+
|           0          |
+-----+
```

15. PERIOD_ADD():

- **Description:** Adds a specified number of months to a period.
- **Syntax:** PERIOD_ADD(period, months)
- **Example:** SELECT PERIOD_ADD(202303, 10);
- **Output:**

```
+-----+
| PERIOD_ADD(202303, 10) |
+-----+
|          202401         |
+-----+
```

16. PERIOD_DIFF():

- **Description:** Calculates the difference in months between two periods.
- **Syntax:** PERIOD_DIFF(period1, period2)
- **Example:** SELECT PERIOD_DIFF(202310, 202303);
- **Output:**

```
+-----+
| PERIOD_DIFF(202310, 202303) |
+-----+
|           7                  |
+-----+
```

17. QUARTER():

- **Description:** Returns the quarter for a given date.

- **Syntax:** QUARTER(date)
- **Example:** SELECT QUARTER('2023-09-15');
- **Output:**

```
+-----+
| QUARTER('2023-09-15') |
+-----+
|           3           |
+-----+ -+
```

18. LAST_DAY():

- **Description:** Returns the last day of the month for a given date.
- **Syntax:** LAST_DAY(date)
- **Example:** SELECT LAST_DAY('2023-09-15');
- **Output:**

```
+-----+
| LAST_DAY('2023-09-15') |
+-----+
| 2023-09-30             |
+-----+
```

19. MAKEDATE():

- **Description:** Creates a date from a year and day of the year.
- **Syntax:** MAKEDATE(year, dayofyear)
- **Example:** SELECT MAKEDATE(2023, 259);
- **Output:**

```
+-----+
| MAKEDATE(2023, 259) |
+-----+
| 2023-09-16          |
+-----+
```

20. MAKETIME():

- **Description:** Creates a time from hours, minutes, and seconds.
- **Syntax:** MAKETIME(hours, minutes, seconds)
- **Example:** SELECT MAKETIME(11, 24, 3);
- **Output:**

```
+-----+
| MAKETIME(11, 24, 3) |
+-----+
| 11:24:03            |
+-----+
```

21. SET_TO_TIME():

- **Description:** Sets the time part of a datetime value.
- **Syntax:** SET_TO_TIME(datetime, time)
- **Example:** SELECT SET_TO_TIME('2023-09-19 23:09:28',

'10:00:00');

• **Output:**

```
+-----+
| SET_TO_TIME('2023-09-19 23:09:28','10:00:00') |
+-----+
| 2023-09-19 10:00:00 |
+-----+
```

22.TIME():

• **Description:** Extracts the time part of a datetime value.

• **Syntax:** TIME(datetime)

• **Example:** SELECT TIME('2023-09-19 23:09:28');

• **Output:**

```
+-----+
| TIME('2023-09-19 23:09:28') |
+-----+
| 23:09:28 |
+-----+
```

23. TIME_FORMAT():

• **Description:** Formats a time value as specified.

• **Syntax:** TIME_FORMAT(time, format)

• **Example:** SELECT TIME_FORMAT('23:09:28', '%h %i %s %p');

• **Output:**

```
+-----+
| TIME_FORMAT('23:09:28', '%h %i %s %p') |
+-----+
| 11 09 28 PM |
+-----+
```

24. TIME_TO_SEC():

• **Description:** Converts a time value to seconds.

• **Syntax:** TIME_TO_SEC(time)

• **Example:** SELECT TIME_TO_SEC('23:09:28');

• **Output:**

```
+-----+
| TIME_TO_SEC('23:09:28') |
+-----+
| 83368 |
+-----+
```

25. TO_DAYS():

• **Description:** Converts a date to days since year 0.

- **Syntax:** TO_DAYS(date)
- **Example:** SELECT TO_DAYS('2023-09-15');
- **Output:**

```
+-----+
| TO_DAYS('2023-09-15') |
+-----+
|          739143 |
+-----+
```

26. WEEK():

- **Description:** Returns the week number for a given date.
- **Syntax:** WEEK(date)
- **Example:** SELECT WEEK(CURDATE());
- **Output:**

```
+-----+
| WEEK(CURDATE()) |
+-----+
|          50 |
+-----+
```

27. WEEKDAY():

- **Description:** Returns the day of the week index for a given date.
- **Syntax:** WEEKDAY(date)
- **Example:** SELECT WEEKDAY(CURDATE());
- **Output:**

```
+-----+
| WEEKDAY(CURDATE()) |
+-----+
|          4 |
+-----+
```

28.WEEKOFYEAR():

- **Description:** Returns the week number for a given date.
- **Syntax:** WEEKOFYEAR(date)
- **Example:** SELECT WEEKOFYEAR(CURDATE());
- **Output:**

```
+-----+
| WEEKOFYEAR(CURDATE()) |
+-----+
|          50 |
+-----+
```

29. YEAR():

- **Description:** Returns the year part of a date or datetime.
- **Syntax:** YEAR(date)
- **Example:** SELECT YEAR('2023-09-15');
- **Output:**

```
+-----+
| YEAR('2023-09-15') |
+-----+
|          2023      |
+-----+
```

30. CAST():

- **Description:** Converts a value from one data type to another.
- **Syntax:** CAST(value AS datatype)
- **Example:** SELECT CAST('20230829' AS DATETIME);
- **Output:**

```
+-----+
| CAST('20230829' AS DATETIME) |
+-----+
| 2023-08-29 00:00:00          |
+-----+
```

31. CONV():

- **Description:** Converts a number from one base to another.
- **Syntax:** CONV(number, from_base, to_base)
- **Example 1:** SELECT CONV(15, 10, 2);
- **Output 1:**

```
+-----+
| CONV(15, 10, 2) |
+-----+
| 1111           |
+-----+
```

- **Example 2:** SELECT CONV(1111, 2, 10);

- **Output 2:**

```
+-----+
| CONV(1111, 2, 10) |
+-----+
| 15                |
+-----+
```

5.Sailor Database

Creating Database:-

```
CREATE DATABASE SAILORDB;
```

```
USE SAILORDB;
```

Create Tables:-

```
CREATE TABLE SAILOR(SNAME VARCHAR(20),SID VARCHAR(10) PRIMARY KEY,SAGE INT,  
                    GENDER CHAR(6) CHECK(GENDER='MALE' OR GENDER='FEMALE'),SALARY INT,  
                    SRATING INT CHECK(SRATING>=1 AND SRATING<=10)
```

```
);
```

```
CREATE TABLE BOAT(BNAME VARCHAR(20),BID VARCHAR(10) PRIMARY KEY,BCOLOR CHAR(10));
```

```
CREATE TABLE RESERVE(SAID VARCHAR(10),FOREIGN KEY(SAID) REFERENCES SAILOR(SID),  
                    BOID VARCHAR(10),FOREIGN KEY(BOID) REFERENCES BOAT(BID),  
                    PRIMARY KEY(SAID,BOID),DAY DATE);
```

Table Descriptions:-

Desc Sailor;

Field	Type	Null	Key	Default	Extra
SNAME	varchar(20)	YES	PRI	NULL	
SID	varchar(10)	NO		NULL	
SAGE	int	YES		NULL	
GENDER	char(6)	YES		NULL	
SALARY	int	YES		NULL	
SRATING	int	YES		NULL	

Desc Boat;

Field	Type	Null	Key	Default	Extra
BNAME	varchar(20)	YES	PRI	NULL	
BID	varchar(10)	NO		NULL	
BCOLOR	char(10)	YES		NULL	

Desc reserve;

Field	Type	Null	Key	Default	Extra
SAID	varchar(10)	NO	PRI	NULL	
BOID	varchar(10)	NO	PRI	NULL	
DAY	date	YES		NULL	

Inserting Data into values:-

```
INSERT INTO BOAT VALUES(  
    'BLACK PEARL','BOAT000','WHITE'),  
    ('OCEAN VOYAGER','BOAT001','RED'),  
    ('WAVE RAIDER','BOAT002','BLUE'),  
    ('SUNSET SERENITY','BOAT003','ORANGE'),  
    ('ISLANDS EXPLORER','BOAT004','INDIGO'),  
    ('CAPTAIN''S PRIDE','BOAT005','GREEN'  
);
```

```
INSERT INTO SAILOR VALUES(  
    'JACK','S0001',40,'MALE',50000,10),  
    ('SPARROW','S0002',40,'MALE',50000,10),  
    ('BARBOSSA','S0003',60,'MALE',70000,9),  
    ('DAVY JONES','S0004',70,'MALE',80000,9),  
    ('SALAZAR','S0005',65,'MALE',75000,8  
);
```

```
INSERT INTO RESERVE VALUES(  
    'S0001','BOAT000','2023-09-23'),  
    ('S0003','BOAT005','2023-09-23'),  
    ('S0001','BOAT004','2023-08-12'),  
    ('S0004','BOAT002','2022-07-21'),  
    ('S0005','BOAT003','2023-04-12');
```

```
Select * From Boat;
```

BNAME	BID	BCOLOR
BLACK PEARL	BOAT000	WHITE
OCEAN VOYAGER	BOAT001	RED
WAVE RAIDER	BOAT002	BLUE
SUNSET SERENITY	BOAT003	ORANGE
ISLANDS EXPLORER	BOAT004	INDIGO
CAPTAIN'S PRIDE	BOAT005	GREEN

Select * from Sailor;

SNAME	SID	SAGE	GENDER	SALARY	SRATING
JACK	S0001	40	MALE	50000	10
SPARROW	S0002	40	MALE	50000	10
BARBOSSA	S0003	60	MALE	70000	9
DAVY JONES	S0004	70	MALE	80000	9
SALAZAR	S0005	65	MALE	75000	8

Select * from reserve;

SAID	BOID	DAY
S0001	BOAT000	2023-09-23
S0001	BOAT004	2023-08-12
S0003	BOAT005	2023-09-23
S0004	BOAT002	2022-07-21
S0005	BOAT003	2023-04-12

LAB-06 IMPLEMENTING THE COMPANY DATABASE

Creation of data base:

```
CREATE DATABASE company;  
  
use company;
```

Creation of Employee table as emp:

```
create table emp(  
    fname VARCHAR(20) NOT NULL,  
    lname VARCHAR(20) NOT NULL,  
    SSN INT PRIMARY KEY,  
    age int check(age>0),  
    sex CHAR(6) CHECK(sex="male" or sex="female"),  
    salary real default 30000,  
    address VARCHAR(50),  
    SSSN INT, DNO INT  
);
```

Creation of department table as dept:

```
create table dept(  
    DNUM INT PRIMARY KEY,  
    dname VARCHAR(10) UNIQUE,  
    dloc VARCHAR(10) NOT NULL,  
    mgrssn INT, sdate DATE,  
    foreign key(mgrssn) references emp(ssn)  
);
```

Adding the foreign key constraints:

```
ALTER TABLE emp add foreign key(sssn) references emp(ssn);  
  
ALTER TABLE emp add foreign key(dno) references dept(dnum);
```

Creation of the project table as project:

```
create table project(  
    pname varchar(15) unique,  
    pnum INT PRIMARY KEY, dno INT,
```

```
foreign key(dno) references dept(dnum)

);
```

Creation of project location as proj_Loc:

```
create table proj_loc(
    pno int, ploc VARCHAR(25),
    primary key(pno,ploc),
    foreign key(pno) references project(pnum)
);
```

Creation of the works on relation as works_on:

```
create table works_on(
    essn int, pno INT, hours int,
    primary key(essn,pno),
    foreign key(essn) references emp(ssn),
    foreign key(pno) references project(pnum)
);
```

Creation of dependent table as dependent:

```
create table dependent(
    essn INT,
    dep_name VARCHAR(15),
    sex char(6) CHECK(sex="male" or sex="female"),
    age int,
    relation varchar(20),
    primary key(essn,dep_name),
    foreign key(essn) references emp(ssn)
);
```

Description of all Tables after the creation :

DESC emp;

Field	Type	Null	Key	Default	Extra
fname	varchar(20)	NO		NULL	
lname	varchar(20)	NO		NULL	
SSN	int	NO	PRI	NULL	
age	int	YES		NULL	
sex	char(6)	YES		NULL	
salary	double	YES		30000	
address	varchar(50)	YES		NULL	
SSSN	int	YES	MUL	NULL	
DNO	int	YES	MUL	NULL	

Desc dept;

Field	Type	Null	Key	Default	Extra
DNUM	int	NO	PRI	NULL	
dname	varchar(10)	YES	UNI	NULL	
dloc	varchar(10)	NO		NULL	
mgrssn	int	YES	MUL	NULL	
sdate	date	YES		NULL	

Desc project;

Field	Type	Null	Key	Default	Extra
pname	varchar(15)	YES	UNI	NULL	
pnum	int	NO	PRI	NULL	
dno	int	YES	MUL	NULL	

Desc proj_loc;

Field	Type	Null	Key	Default	Extra
pno	int	NO	PRI	NULL	
ploc	varchar(25)	NO	PRI	NULL	

Desc works_on;

Field	Type	Null	Key	Default	Extra
essn	int	NO	PRI	NULL	
pno	int	NO	PRI	NULL	
hours	int	YES		NULL	

Desc dependent;

Field	Type	Null	Key	Default	Extra
essn	int	NO	PRI	NULL	
dep_name	varchar(15)	NO	PRI	NULL	
sex	char(6)	YES		NULL	
age	int	YES		NULL	
relation	varchar(20)	YES		NULL	

Insertion of data:

10) Inserting into emp table:

```
INSERT INTO emp (fname, lname, SSN, age, sex, salary, address)
```

```
VALUES
```

```
('John', 'Doe', 1000,34, 'male', 50000, 'Mumbai'),
('Jane', 'Smith', 1001,32, 'female', 45000, 'Bangalore'),
('Amit', 'Kumar', 1002, 39,'male', 55000, 'Delhi'),
('Priya', 'Sharma', 1003, 30, 'female', 48000, 'Kolkata'),
('Rajesh', 'Patel', 1004, 36, 'male', 52000, 'Chennai'),
('Sara', 'Gupta', 1005, 28,'female', 47000, 'Hyderabad'),
('Rahul', 'Singh', 1006, 42,'male', 53000, 'Mumbai'),
('Anita', 'Verma', 1007, 37,'female', 49000, 'Delhi'),
('Sanjay', 'Yadav', 1008, 33,'male', 51000, 'Bangalore'),
('Meera', 'Das', 1009, 29, 'female', 46000, 'Kolkata'),
('Vijay', 'Raj', 1010, 43 , 'male', 54000, 'Chennai'),
('Rina', 'Shah', 1011, 38 ,'female', 50000, 'Hyderabad');
```

11) Inserting data into dept table:

```
INSERT INTO dept (DNUM, dname, dloc, mgrssn, sdate)
```

```
VALUES
```

```
(100, 'HR', 'Mumbai', 1003, '2023-09-01'),
(102, 'Engg', 'Delhi', 1001, '2023-09-10'),
(103, 'Marketing', 'Kolkata', 1010, '2023-08-28'),
(105, 'Research', 'Hyderabad', 1002, '2023-08-20');
```

Now we update the respective departments in the emp table

```
UPDATE emp SET ssn = '1003', dno = '100' WHERE (ssn = '1000');  
UPDATE emp SET dno = '102' WHERE (ssn = '1001');  
UPDATE emp SET dno = '105' WHERE (ssn = '1002');  
UPDATE emp SET dno = '100' WHERE (ssn = '1003');  
UPDATE emp SET ssn = '1001', dno = '102' WHERE (ssn = '1004');  
UPDATE emp SET ssn = '1001', dno = '102' WHERE (ssn = '1005');  
UPDATE emp SET ssn = '1002', dno = '105' WHERE (ssn = '1006');  
UPDATE emp SET ssn = '1001', dno = '102' WHERE (ssn = '1007');  
UPDATE emp SET ssn = '1010', dno = '103' WHERE (ssn = '1008');  
UPDATE emp SET dno = '103' WHERE (ssn = '1010');  
UPDATE emp SET ssn = '1010', dno = '103' WHERE (ssn = '1011');
```

12) Inserting the data into the Project table:

```
INSERT INTO project (pname, pnum, dno)VALUES  
('Market inc', 304, 103),  
('Folding watch',301, 105),  
('Emp training',302, 100),  
('watch', 303, 102);
```

13) Inserting the data into proj_loc table:

```
insert into proj_loc values  
(301,'Hyderabad'), (302,'Bengalore'),  
(302,'Hyderabad'),(303,'Bengalore');
```

14) Inserting data into the works_on table:

```
insert into works_on values  
(1001,303,25),  
(1004,303,30),  
(1005,303,30),  
(1002,301,20),  
(1006,301,20),  
(1000,302,18),  
(1003,302,20),  
(1010,304,20),  
(1008,304,20);
```

15) Inserting the data into the dependent table:

```
INSERT INTO dependent (essn, dep_name, sex, age, relation) VALUES
```

```
(1001, 'varun', 'male', 12, 'Child'),
(1001, 'Sarah', 'female', 8, 'Child'),
(1002, 'Emily', 'female', 32, 'Spouse'),
(1003, 'Michael', 'male', 10, 'Child'),
(1004, 'Lilly', 'female', 4, 'Child'),
(1005, 'Michael', 'male', 31, 'Spouse'),
(1006, 'Daniel', 'male', 7, 'Child'),
(1007, 'Sophia', 'female', 6, 'Child');
```

Retrieving the data from the all records from every Tables:

DEPT: select * from emp;

DNUM	dname	dloc	mgrssn	sdate
100	HR	Mumbai	1003	2023-09-01
102	Engg	Delhi	1001	2023-09-10
103	Marketing	Kolkata	1010	2023-08-28
105	Research	Hyderabad	1002	2023-08-20

EMP: select * from emp;

fname	lname	SSN	age	sex	salary	address	SSSN	DNO
John	Doe	1000	34	male	50000	Mumbai	1003	100
Jane	Smith	1001	32	female	45000	Bangalore	NULL	102
Amit	Kumar	1002	39	male	55000	Delhi	NULL	105
Priya	Sharma	1003	30	female	48000	Kolkata	NULL	100
Rajesh	Patel	1004	36	male	52000	Chennai	1001	102
Sara	Gupta	1005	28	female	47000	Hyderabad	1001	102
Rahul	Singh	1006	42	male	53000	Mumbai	1002	105
Anita	Verma	1007	37	female	49000	Delhi	1001	102
Sanjay	Yadav	1008	33	male	51000	Bangalore	1010	103
Meera	Das	1009	29	female	46000	Kolkata	1010	103
Vijay	Raj	1010	43	male	54000	Chennai	NULL	103
Rina	Shah	1011	38	female	50000	Hyderabad	1010	103

Project: select * from project;

pname	pnum	dno
Folding watch	301	105
Emp training	302	100
watch	303	102
Market inc	304	103

Proj_loc: select * from proj_loc;

pno	ploc
301	Hyderabad
302	Bengalore
302	Hyderabad
303	Bengalore

Works_on : select * from works_on;

essn	pno	hours
1000	302	18
1001	303	25
1002	301	20
1003	302	20
1004	303	30
1005	303	30
1006	301	20
1008	304	20
1010	304	20

Dependent : Select * from dependent;

essn	dep_name	sex	age	relation
1001	Sarah	female	8	Child
1001	varun	male	12	Child
1002	Emily	female	32	Spouse
1003	Michael	male	10	Child
1004	Lilly	female	4	Child
1005	Michael	male	31	Spouse
1006	Daniel	male	7	Child
1007	Sophia	female	6	Child

LAB-07 DCL and TCL Commands

DCL (Data Control Language)

Data Control Language is used to access the stored data. DCL includes commands such as GRANT and REVOKE which mainly deals with rights, permissions and other controls of the database system.

GRANT :- It is used to give user access privileges to a database.

Syntax : GRANT <privileges list>

ON <tables/views>

TO user(s);

Example: Displaying default user:-

>SELECT user();

```
mysql> select user();
+-----+
| user() |
+-----+
| root@localhost |
+-----+
1 row in set (0.00 sec)
```

Displaying user, host:-

>SELECT user, host FROM MYSQL.USER;

```
mysql> select user, host from mysql.user;
+-----+-----+
| user          | host      |
+-----+-----+
| mysql.infoschema | localhost |
| mysql.session   | localhost |
| mysql.sys       | localhost |
| practice        | localhost |
| root           | localhost |
| sample          | localhost |
+-----+-----+
6 rows in set (0.00 sec)
```

Create a new user :-

CREATE USER username IDENTIFIED BY password;

EX :- >CREATE USER 'new'@localhost IDENTIFIED BY 'dbms';

>SELECT user, host FROM MYSQL.USER;

```
mysql> select user, host from mysql.user;
+-----+-----+
| user          | host      |
+-----+-----+
| mysql.infoschema | localhost |
| mysql.session   | localhost |
| mysql.sys       | localhost |
| new            | localhost |
| practice        | localhost |
| root           | localhost |
| sample          | localhost |
+-----+-----+
7 rows in set (0.00 sec)
```

Switching from one user to another user:-

SYSTEM MYSQL -u USERNAME -p ;

Ex:- >SYSTEM MYSQL -u new -p;

Enter password :- ****

Grant SELECT permissions to new user:-

>GRANT SELECT ON sailors.* TO 'new'@localhost;

>SELECT sailor.sname FROM sailor;

```
mysql> system mysql -u new -p;
Enter password: ****
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 15
Server version: 8.0.34 MySQL Community Server - GPL

Copyright (c) 2000, 2023, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> use sailordatabase;
Database changed
mysql> select sailor.sname from sailor;
+-----+
| sname |
+-----+
| rahul |
| ramarao |
| bob |
| priya |
| ram |
+-----+
5 rows in set (0.00 sec)
```

Grant INSERT permissions to new user:-

>GRANT INSERT,UPDATE ON sailors.* TO 'new'@localhost;

>SELECT sailor.sname FROM sailor;

```
mysql> use sailordatabase;
Database changed
mysql> insert into sailor values('praveen',10,9,12,'vijayawada');
Query OK, 1 row affected (0.17 sec)

mysql> select sailor.sname from sailor;
+-----+
| sname |
+-----+
| rahul |
| ramarao |
| bob |
| priya |
| ram |
| praveen |
+-----+
6 rows in set (0.00 sec)
```

Grant all privileges same as root:-

>GRANT ALL PRIVILEGES ON *.* TO 'new'@localhost WITH GRANT OPTION;

To view grants of user:-

>SHOW GRANTS FOR 'new'@localhost;

```
mysql> show grants for 'new'@localhost;
+-----+
| Grants for new@localhost |
+-----+
| GRANT USAGE ON *.* TO `new`@`localhost` |
| GRANT SELECT, INSERT, UPDATE ON `sailordatabase`.* TO `new`@`localhost` |
+-----+
2 rows in set (0.00 sec)
```

REVOKE :-

The REVOKE statement can be used to remove granted permissions, and the DENY statement can be used to prevent a principal from gaining a specific permission through a GRANT. It is used to revoke previously granted privileges. Granting a permission removes DENY or REVOKE of that permission on the specified securable.

SYNTAX :- REVOKE <privileges_list>

ON <tables>

FROM user(s);

EXAMPLE :- Revoking permissions from 'new'

>REVOKE SELECT ON db.customers FROM 'new'@localhost;

```
mysql> revoke select on db.customers from 'new';
Query OK, 0 rows affected (0.03 sec)
```

EX :-

>SHOW GRANTS for new;

```
mysql> show grants for new;
+-----+
| Grants for new@% |
+-----+
| GRANT USAGE ON *.* TO `new`@`%` |
| GRANT INSERT ON `db`.`customers` TO `new`@`%` |
+-----+
2 rows in set (0.00 sec)
```

TCL COMMANDS

TCL is used to manage transactions in a database. Transactions are sequences of one or more SQL statements that are executed as a single unit.

There are 3 basic commands in TCL.

1.COMMIT

2.SAVEPOINT

3.ROLLBACK

1)COMMIT:

Used to save all changes during the current transaction.It makes changes permanent and finalizes the transaction.

Set autocommit to zero to work with TCL commands they are stored temporarily

Example:

We can demonstrate commit by inserting a value while autocommit is zero so that value is not inserted

Initial Table :Table name :Customers

```
>select * from customers;
```

```
mysql> select * from customers;
```

id	name	age	address	salary
100	pavan sai	19	tiruvuru	1300
101	jhansi	20	vizag	1200
102	varshini	21	guntur	1500
103	heera	18	vizag	1000
104	shamitha	20	kerala	1100
105	bhavani	21	hyderabad	1000
106	john	20	tiruvuru	1100

```
>set autocommit=0;
```

```
>insert into customers values(107,"ujwala",20,"vizag",1300);
```

Checking table whether value inserted or not

```
>select * from customers;
```

```
mysql> select * from customers;
```

id	name	age	address	salary
100	pavan sai	19	tiruvuru	1300
101	jhansi	20	vizag	1200
102	varshini	21	guntur	1500
103	heera	18	vizag	1000
104	shamitha	20	kerala	1100
105	bhavani	21	hyderabad	1000
106	john	20	tiruvuru	1100
107	ujwala	22	vizag	1000

If we close the system check for the customers we cannot find changes made(i.e.,row is not inserted permanently)

```
>select * from customers;
```

```
mysql> select * from customers;
```

id	name	age	address	salary
100	pavan sai	19	tiruvuru	1300
101	jhansi	20	vizag	1200
102	varshini	21	guntur	1500
103	heera	18	vizag	1000
104	shamitha	20	kerala	1100
105	bhavani	21	hyderabad	1000
106	john	20	tiruvuru	1100

To avoid this we can put commit statement at end of program to make changes permanent

```
>COMMIT;
```

2)SAVE POINT:

Creates a point within a transaction to which we can rollback later.

Allows us to partially undo changes made in the transaction up to the savepoint.

Syntax:SAVEPOINT SavePointName;

Example:

```
>SAVEPOINT sp1;
```

Updating some operations on customers table

```
>update customers set salary=2000 where name="ujwala";
```

To see updates:

>SELECT * from customers;

```
mysql> select * from customers;
+-----+-----+-----+-----+-----+
| id | name      | age | address | salary |
+-----+-----+-----+-----+-----+
| 100 | pavan sai | 19  | tiruvuru | 1300   |
| 101 | jhansi    | 20  | vizag    | 1200   |
| 102 | varshini  | 21  | guntur   | 1500   |
| 103 | heera     | 18  | vizag    | 1000   |
| 104 | shamitha  | 20  | kerala   | 1100   |
| 105 | bhavani   | 21  | hyderabad | 1000   |
| 106 | john      | 20  | tiruvuru | 1100   |
| 107 | ujwala    | 22  | vizag    | 2000   |
+-----+-----+-----+-----+-----+
```

3)ROLLBACK:

Roll Back undo the transactions after savepoint

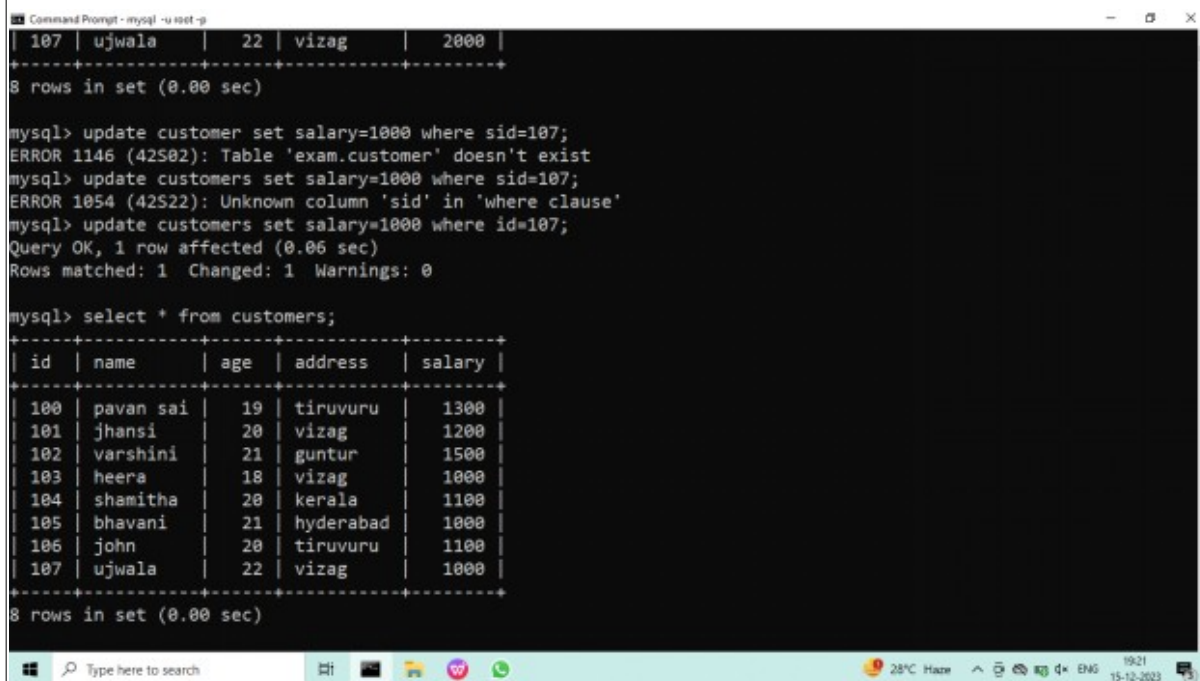
Syntax:ROLLBACK TO SavePointName;

Example:

Rollback the changes done by sp1

>ROLLBACK TO SP1;

>SELECT * FROM Customers;



```
Command Prompt - mysql -u root -p
+-----+-----+-----+-----+-----+
| 107 | ujwala    | 22  | vizag    | 2000   |
+-----+-----+-----+-----+-----+
8 rows in set (0.00 sec)

mysql> update customer set salary=1000 where sid=107;
ERROR 1146 (42S02): Table 'exam.customer' doesn't exist
mysql> update customers set salary=1000 where sid=107;
ERROR 1054 (42S22): Unknown column 'sid' in 'where clause'
mysql> update customers set salary=1000 where id=107;
Query OK, 1 row affected (0.06 sec)
Rows matched: 1  Changed: 1  Warnings: 0

mysql> select * from customers;
+-----+-----+-----+-----+-----+
| id | name      | age | address | salary |
+-----+-----+-----+-----+-----+
| 100 | pavan sai | 19  | tiruvuru | 1300   |
| 101 | jhansi    | 20  | vizag    | 1200   |
| 102 | varshini  | 21  | guntur   | 1500   |
| 103 | heera     | 18  | vizag    | 1000   |
| 104 | shamitha  | 20  | kerala   | 1100   |
| 105 | bhavani   | 21  | hyderabad | 1000   |
| 106 | john      | 20  | tiruvuru | 1100   |
| 107 | ujwala    | 22  | vizag    | 1000   |
+-----+-----+-----+-----+-----+
8 rows in set (0.00 sec)
```

LAB-08 Set Operations and Queries on Sailor Database

Set comparison operators:-

Set comparison operators in SQL are used to compare a value or a Set of values to another set of values in the database.

1) IN operator:

It is used to compare a value to a list of values. It checks if the value exists in the list.

Syntax:- Select col1, Col2,... from table name where col in (val1, val2,...);

1) Retrieve names of sailors who reserved boat with Id 'Boat003'?

>Select Sname from sailor where sid in (select said from reserve where boid='Boat003');

Sname
SALAZAR

1) Retrieve names of sailors who reserved red boat ?

> Select sname from Sailor where sid in (Select said from reserve, boat where Bid=boid and Bcolor='red');

sname
JACK
SPARROW
DAVY JONES
SALAZAR

2) Not in operator:-

Not in operator is negation of in. It checks if a value does not exist in list of values

Syntax:- Select A1, A2,... from T_name where col not in (v1,v2,...);

1) Retrieve names of sailors who reserved a red but not green boat?

>select sname from sailor where sid in (select sid from reserve, boat Where Bid=Boid and Bcolor="red") and Sid not in (Select said from reserve, boat where Bid=Boid and Bcolor='green');

sname
JACK
DAVY JONES
SALAZAR

3) Exists operator:- It is used to check if a sub-Query returns any rows If yes, Condition is true.

Syntax:-Select C1,C2,... from T_name where exists (select * from T where <Condition>);

1) Retrieve names of sailors who reserved boat no: 'Boat005'

> Select s.sname from sailor s where exists (select * from reserve r where S.sid=r.said and r.boid='boat005');

sname
SPARROW
BARBOSSA

2) Retrieve names of sailors who reserved red boat?

> select s.sname from sailor s where exists (select * from reserve r, Boat b where s.sid=r.said and r.boid=b.bid and b.bcolor='red');

sname
SALAZAR
JACK
SPARROW
DAVY JONES

4)Not exists operator:-

Not exists operator is negation of exists. It checks if sub-query returns no rows or not

Syntax:- Select C1,C2,.. from t_name where not exists (select * from t_name where <condition>);

1)Retrieve names of sailors who hasn't reserved 'red' boat?

> Select S.Sname from sailor s where not exists (Select * from reserve r, Boat b where s.sid=r.said and r.boid=b.bid and b.bcolor='red');

sname
BARBOSSA

5) Any operator:-

Any operator is used to compare a single value with a set of values returned by a sub query. It returns True if the compare is True for at least one value in set

Syntax:-

Select C1,C2,... from t_name where c_name operator any (Sub query);

1) Retrieve names of Sailors whose rating is better than sailor 'Barbossa' ?

> Select sname from sailor where srating> any (Select Srating from Sailor where Sname='Barbossa');

sname
JACK
SPARROW

6) All operator:-

All operator is used to compare a single valve with a set of values returned by a Sub-query. It returns True if comparison is True for all values in set

Syntax:- Select C1, C2,... from T_name where c_name operator all (Sub-query);

1) Retrieve names of sailors who has highest rating?

> Select s.sname from sailor s where S.Srating >= all (Select Srating from sailor);

sname
JACK
SPARROW

Set Manipulation operations:-

1)Union:-

It is used to combine two sets and it returns union of two given relations. (It follows Union Compatibility)

Syntax:- select c1,c2,... from T1 union Select (c11,c22,.. from T2);

1) Retrieve names of sailors who reserve either red or green boat?

select Distinct s.Sname from Sailor s, Reserve r, Boat B where s.sid=r.Said and r.boid=b.bid and b.Bcolor='red' Union Select Distinct s1.sname from Sailor S1,reserve r1, Boat B1 where s1.Sid=r1.Said and r1.boid=b1.bid and b1.bcolor='green';

Sname
SALAZAR
JACK
SPARROW
DAVY JONES
BARBOSSA

2) Intersect:- It is used to combine two Select statement's but returned result will be intersection of two relations of two select commands.

Syntax:- Select C1,C2,... from T1 where <condition> Intersect Select C11, C22,.. from T2 where <Condition>

*Mysql doesn't support Intersect Clause, we can implement it using 'and' operator.

1) Retrieve names of sailors who reserve both red and green boats?

> Select s.sname from sailor s, reserve r1, boat b1, reserve r2, boat b2, where s.sid=r1.said and r1.boid=b1.bid and b1.bcolor='red' and S.Sid=r2.said and r2.boid=b2.bid and b2.bcolor="green";

sname
SPARROW

3)Minus:-

It is used to subtract result set obtained by first Select query from result set obtained by second select Query.

Syntax:-select c1,c2,... from t1 where <condition> minus select c11,c22,... from t2 where <condition>

* Like intersect, Minus operator is also not supported by Mysql, instead use 'in', 'not in' and 'and' operators Combined.

1) Retrieve names of sailors whose reserve green but not red boat?

> Select Sname from sailor where sid in (select said from reserve r1, boat b1 where r1.boid=b1.bid and b1.bcolor='green') and Sid not in (Select Said from reserve r2, boat b2 where r2.boid=b2.bid and b2.bcolor='red');

Sname
BARBOSSA

4)Cross product:-

It returns cross product of two sets (or) returns tuples from two relations

Syntax: select c1,c2,.. from Table1, Table2 where <condition>;

1) Retrieve names of sailors who reserved boat with Id 'Boat005'?

> Select s.sname from Sailor s, reserve r where s.sid= r.Said and r.boid= 'Boat003';

sname
SALAZAR

Aggregation functions:-

Aggregation operators in SQL are used to perform calculations on sets of values in a column to produce a single result. They are often applied in conjunction with the 'Group By' clause to group data into categories before applying the aggregation functions:

1) Count:-Count operator is used to count no of rows or non-null values in a column.

Syntax: Select count (column-name) from table_name;

1)Count Distinct ages of sailors?

Select count (Distinct sage) from Sailor as age;

age
4

2) Sum:- Sum Operator is used to count the Sum of values in a numeric Column

Syntax:- Select Sum (C1) from table_name;

3)Avg:- Avg operator is used to calculate average of values in a numeric column

Syntax:- Select avg (c1) from table_name;

1) Find average age of sailors with rating 10?

> select avg (sage) from Sailor where srating=10;

avg (sage)
40.0000

4)Min:- Min function is used to find minimum value in a column.

Syntax:- Select min (C1) from table_name;

1) find age of Youngest Sailor who is eligible to vote for each rating level with almost 2 Such Sailors?

> Select Srating, min (Sage) from Sailor Group By Srating Having Count(*)>=2;

Srating	min(Sage)
10	40
9	60

5)Max:- Max function is used to find the maximum in a numeric column

Syntax:- select max(C1) from T1;

1) Find name and age of oldest Sailor.

> Select sname, sage from Sailor where sage in (select max(sage) from Sailor);

sname	sage
DAVY JONES	70

2) Select names of Sailors who are older than oldest sailor whose rating is 10?

> Select name from Sailor where sage> All (Select sage from Sailor Where Srating = 10);

sname
BARBOSSA
DAVY JONES
SALAZAR

3) for each red boat find number of reservations?

> Select B.Bid, Count(*) from Boat B, Reserve R where R.Boid=B.Bid and B.Bcolor="red" Group By B.Bid;

Bid	Count(*)
BOAT001	1
BOAT004	3

4) Find average age of sailors who are Eligible for voting in each rating level that has atleast 2 sailors

> Select avg (sage) from Sailor where Sage>=18 Group By Srating Having count(*)>=2;

avg (sage)
40.0000
65.0000

LAB-09 QUERIES ON THE COMPANY DATABASE

8. Retrieve the details of employee who are working in department 100

```
select * from emp where dno=100;
```

fname	lname	SSN	age	sex	salary	address	SSSN	DNO
John	Doe	1000	34	male	50000	Mumbai	1003	100
Priya	Sharma	1003	30	female	48000	Kolkata	NULL	100

9. Retrieve the details of employees who are working in the department number 105 having salary more than 30,000

```
select * from emp where dno=105 and salary>30000;
```

fname	lname	SSN	age	sex	salary	address	SSSN	DNO
Amit	Kumar	1002	39	male	55000	Delhi	NULL	105
Rahul	Singh	1006	42	male	53000	Mumbai	1002	105

10. Retrieve the details of Female employers who works in department number 105 whose salary greater than 40,000

```
select * from emp where dno=103 and sex='female' and salary>40000;
```

fname	lname	SSN	age	sex	salary	address	SSSN	DNO
Meera	Das	1009	29	female	46000	Kolkata	1010	103
Rina	Shah	1011	38	female	50000	Hyderabad	1010	103

11. Select Fname ,Lname of all employees

```
select fname, lname from emp;
```

fname	lname
John	Doe
Jane	Smith
Amit	Kumar
Priya	Sharma
Rajesh	Patel
Sara	Gupta
Rahul	Singh
Anita	Verma
Sanjay	Yadav
Meera	Das
Vijay	Raj
Rina	Shah

12. Retrieve the fname, lname and salary of female employees who working in department 102

select fname,lname,salary from emp where sex='female' and dno=102;

fname	lname	salary
Jane	Smith	45000
Sara	Gupta	47000
Anita	Verma	49000

13. Retrieve SSN, address of male employees in salary greater than 40,000 and work in Department 103

select ssn,address from emp where salary>40000 and dno=103 and sex='male';

ssn	address
1008	Bangalore
1010	Chennai

14. Retrieve names of employees who have dependents

select fname,lname from emp cross join dependent on ssn=essn;

fname	lname
Jane	Smith
Jane	Smith
Amit	Kumar
Priya	Sharma
Rajesh	Patel
Sara	Gupta
Rahul	Singh
Anita	Verma

15. Retrieve the names of the employees who works on all projects of amit kumar

select e.fname,e.lname from emp e where not exists(
select * from project p where not exists (
select w.pno from works_on w where w.essn=e.ssn and w.pno=p.pnum) and
p.pnum in (select w.pno from works_on w,emp e1 where e1.ssn=w.essn and e1.fname='amit' and
e1.lname='kumar'));

fname	lname
Amit	Kumar
Rahul	Singh

16. Retrieve names of employees who works on all projects controlled by department 103

```
select e.fname,e.lname from emp e where  
not exists (select pnum from project where dno=103 and  
pnum not in(select pno from works_on w where e.ssn=w.essn));
```

fname	lname
Sanjay	Yadav
Vijay	Raj

17. Retrieve name ,salary ,address of all employees who are working in HR Department

```
select fname,lname,salary,address from emp join dept on dno=dnum where dname='HR';
```

fname	lname	salary	address
John	Doe	50000	Mumbai
Priya	Sharma	48000	Kolkata

18. Retrieve the names of employees who have no dependents

```
select concat(fname," ",lname) as name from emp where ssn not in (select essn from dependent);
```

name
John Doe
Sanjay Yadav
Meera Das
Vijay Raj
Rina Shah

19. Retrieve the names of employee Managers who have atleast one dependent

```
select concat(fname," ",lname) as name from emp join dept on ssn=mgrssn where ssn in (select  
essn from dependent);
```

name
Jane Smith
Amit Kumar
Priya Sharma

20. Retrieve number of dependents for each employee

select essn,count(*) as count from dependent group by essn;

essn	count
1001	2
1002	1
1003	1
1004	1
1005	1
1006	1
1007	1

21. Increase the salary of all employees of department 103 by 25% hike

select fname,lname,salary+salary*0.25 as sal from emp where dno=103;

fname	lname	sal
Sanjay	Yadav	63750
Meera	Das	57500
Vijay	Raj	67500
Rina	Shah	62500

22. For every project located in Hyderabad Retrieve project number, controlling department number, controlling department manager , last name ,Address and salary.

select pnum,dnum,fname,lname,address,salary from proj_loc pl,project p,dept d,emp e
where pl.pno=p.pnum and p.dno=d.dnum and d.mgrssn=e.ssn and ploc='Hyderabad';

pnum	dnum	fname	lname	address	salary
301	105	Amit	Kumar	Delhi	55000
302	100	Priya	Sharma	Kolkata	48000

23. List the names of all employees at least two dependents

select fname,lname from emp join dependent on essn=ssn group by essn having count(*)>=2;

fname	lname
Jane	Smith

24. Retrieve the list of projects that involves Amit as a worker or as a manager

```
select pno from works_on where essn in (select ssn from emp where fname='Amit') union  
select pnum from project where dno in (select dnum from dept where mgrssn =(select ssn from  
emp where fname='Amit'));
```

pno
301

25. Show the resulting salaries and names of employees who are working to the project watch is given 20% hike

```
select concat(fname," ",lname) as name,salary ,salary*0.25 as increment from  
emp,works_on,project where essn=ssn and pno=pnum and pname='Watch';
```

name	salary	increment
Jane Smith	45000	11250
Rajesh Patel	52000	13000
Sara Gupta	47000	11750

LAB 10 VIEWS AND TRIGGERS

VIEWS :

A view is a virtual table based on the result-set of an SQL statement.

A view contains rows and columns, just like a real table. The fields in a view are fields from one or more real tables in the database.

You can add SQL statements and functions to a view and present the data as if the data were coming from one single table.

A view is created with the CREATE VIEW statement.

CREATE VIEW Syntax :

CREATE VIEW view_name AS

SELECT column1, column2, ...

FROM table_name

WHERE condition;

EXAMPLE :

CREATE VIEW best_sailors AS SELECT * FROM sailor WHERE srating >= 9;

select * from best_sailors;

OUTPUT :

```
mysql> select * from best_sailors;
+-----+-----+-----+-----+-----+-----+
| SNAME | SID | SAGE | SEX | SALARY | SRATING |
+-----+-----+-----+-----+-----+-----+
| DEEPA | 8 | 35 | FEMALE | 4900 | 10 |
| GEETHA | 24 | 32 | FEMALE | 4900 | 10 |
| ANVITHA | 43 | 23 | FEMALE | 9000 | 10 |
| SURESH | 87 | 26 | MALE | 7500 | 10 |
+-----+-----+-----+-----+-----+-----+
4 rows in set (0.00 sec)
```

MySQL

Updating a View :

A view can be updated with the CREATE OR REPLACE VIEW statement.

Update View :

The SQL UPDATE VIEW command can be used to modify the data of a view.

All views are not updatable. So, UPDATE command is not applicable to all views. An updatable view is one which allows performing a UPDATE command on itself without affecting any other table.

When can a view be updated?

1. The view is defined based on one and only one table.
2. The view must include the PRIMARY KEY of the table based upon which the view has been created.
3. The view should not have any field made out of aggregate functions.
4. The view must not have any DISTINCT clause in its definition.
5. The view must not have any GROUP BY or HAVING clause in its definition.
6. The view must not have any SUBQUERIES in its definitions.
7. If the view you want to update is based upon another view, the later should be updatable.
8. Any of the selected output fields (of the view) must not use constants, strings or value expressions.

EXAMPLE :

-- Updatable only on a single table and only if it has primary key and not null constraints

update best_sailors set salary = 8500 where sname = 'suresh';

OUTPUT :

```
mysql> update best_sailors set salary = 8500 where sname = 'suresh';
Query OK, 1 row affected (0.06 sec)
Rows matched: 1  Changed: 1  Warnings: 0
```

select * from best_sailors where sname = 'suresh'; *-- updated*

OUTPUT :

```
mysql> select * from best_sailors where sname = 'suresh';
+-----+-----+-----+-----+-----+-----+
| SNAME | SID  | SAGE | SEX  | SALARY | SRATING |
+-----+-----+-----+-----+-----+-----+
| SURESH | 87   | 26   | MALE | 8500   | 10      |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

select * from sailor where sname = 'suresh'; *-- updated in base table also*

OUTPUT :

```
mysql> select * from sailor where sname = 'suresh';
+-----+-----+-----+-----+-----+-----+
| SNAME | SID  | SAGE | SEX  | SALARY | SRATING |
+-----+-----+-----+-----+-----+-----+
| SURESH | 87   | 26   | MALE | 8500   | 10      |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

*-- Insert values
that does not
satisfy the
condition of the
view*

insert into best_sailors values

('VINEESHA',56,25,'FEMALE',8000,10),

('ARUN',55,21,'MALE',7000,9);

OUTPUT :

```
mysql> insert into best_sailors values
->      ('VINEESHA',56,25,'FEMALE',8000,10),
->      ('ARUN',55,21,'MALE',7000,9);
Query OK, 2 rows affected (0.07 sec)
Records: 2  Duplicates: 0  Warnings: 0
```

select * from best_sailors where sid = 56 or sid = 55; *# Did not accept which does not come under the condition*

OUTPUT:

```
mysql> select * from best_sailors where sid = 56 or sid = 55;
+-----+-----+-----+-----+-----+-----+
| SNAME  | SID | SAGE | SEX   | SALARY | SRATING |
+-----+-----+-----+-----+-----+-----+
| VINEESHA | 56 | 25 | FEMALE | 8000 | 10 |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

select * from sailor where sid = 56 or sid = 55; *# Both are added in the base table*

OUTPUT :

```
mysql> select * from sailor where sid = 56 or sid = 55;
+-----+-----+-----+-----+-----+-----+
| SNAME  | SID | SAGE | SEX   | SALARY | SRATING |
+-----+-----+-----+-----+-----+-----+
| ARUN    | 55 | 21 | MALE   | 7000 | 9 |
| VINEESHA | 56 | 25 | FEMALE | 8000 | 10 |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

--
*Delete
values
from
the
view*

delete from best_sailors where sname = 'suresh';

OUTPUT :

```
mysql> delete from best_sailors where sname = 'suresh';
Query OK, 1 row affected (0.09 sec)
```

select * from best_sailors where sname = 'suresh' or sid = 8;

OUTPUT :

```
mysql> select * from best_sailors where sname = 'suresh' or sid = 8;
+-----+-----+-----+-----+-----+-----+
| SNAME | SID | SAGE | SEX   | SALARY | SRATING |
+-----+-----+-----+-----+-----+-----+
| DEEPA | 8   | 35   | FEMALE | 4900   | 10      |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

select * from sailor where sname = 'suresh' or sid = 8; -- Deleted from base table also

OUTPUT :

```
mysql> select * from sailor where sname = 'suresh' or sid = 8;
+-----+-----+-----+-----+-----+-----+
| SNAME | SID | SAGE | SEX   | SALARY | SRATING |
+-----+-----+-----+-----+-----+-----+
| DEEPA | 8   | 35   | FEMALE | 4900   | 10      |
+-----+-----+-----+-----+-----+-----+
1 row in set (0.00 sec)
```

Using aggregation functions

create view aggregate_sailors as select srating, count(*) as counts from sailor group by srating;

OUTPUT :

```
mysql> create view aggregate_sailors as select srating, count(*) as counts from sailor group by srating;
Query OK, 0 rows affected (0.07 sec)
```

select * from aggregate_sailors order by srating;

OUTPUT :

```
mysql> select * from aggregate_sailors order by srating;
+-----+-----+
| srating | counts |
+-----+-----+
| 1       | 1      |
| 2       | 1      |
| 3       | 2      |
| 4       | 3      |
| 5       | 2      |
| 6       | 5      |
| 7       | 3      |
| 8       | 5      |
| 9       | 5      |
| 10      | 4      |
+-----+-----+
10 rows in set (0.00 sec)
```

insert
into

aggregate_sailors values (11,3); # not insertable

OUTPUT :

```
mysql> insert into aggregate_sailors values (11,3);
ERROR 1471 (HY000): The target table aggregate_sailors of the INSERT
is not insertable-into
```

update aggregate_sailors set counts = 3 where srating = 1; # not updatable

OUTPUT :

```
mysql> update aggregate_sailors set counts = 3 where srating = 1;
ERROR 1288 (HY000): The target table aggregate_sailors of the UPDATE
is not updatable
```

-- Multiple tables

create view multiple as select distinct s.sname, r.bid from sailor s, reserves r where s.srating = 8 and s.sid = r.sid;

OUTPUT :

```
mysql> create view multiple as select distinct s.sname, r.bid from s
ailor s, reserves r where s.srating = 8 and s.sid = r.sid;
Query OK, 0 rows affected (0.04 sec)
```

select * from multiple;

insert into multiple value ('HARINI',6534);

delete from multiple where bid = 1234;

OUTPUT :

```
mysql> select * from multiple;
+-----+-----+
| sname  | bid  |
+-----+-----+
| YASHU  | 1234 |
| YASHU  | 4531 |
| YASHU  | 7569 |
| LAKSHMI | 3123 |
+-----+-----+
4 rows in set (0.00 sec)

mysql> insert into multiple value ('HARINI',6534);
ERROR 1471 (HY000): The target table multiple of the INSERT is not
insertable-into
mysql> delete from multiple where bid = 1234;
ERROR 1288 (HY000): The target table multiple of the DELETE is not
updatable
```

TRIGGERS:-

It is a Special type of stored procedure that is invoked automatically in response to an event.

Each trigger is associated with a table which activates any one statement such as Insert, Delete, Update.

Types of Triggers:

There are six types of actions or events in form of

- 1) Before Insert: It is activated before the insertion of data into table.
- 2) After Insert: It is activated after the insertion of data into table.
- 3) Before Update: It is activated before the update of data in table.
- 4) After Update: It is activated after the update of data in table.
- 5) Before Delete: It is activated before the data removed from table.
- 6) After Delete: It is activated after data removed from table.

Syntax:

Create trigger trigger_name

(After/Before) (Insert/ Update Delete)

on table-name for each row

Begin

-----Variable declarations

-----Trigger code.

End;

/

Queries on Triggers:-

1) Let us create a trigger which keeps track of number of employees inserted into employee table.

create trigger count_incre

after insert on Emp

```

for each row
begin
    update emps_count set count=count+1;
end;
/
create table emp_count(count int);
insert into emp_count values(0);
insert into emp values("surya","arjun",171,27,'f',25000,'egypt',5,null);
select * from emps_count;

```

OUTPUT:-

```

mysql> select * from emps_count;
-> select * from emps_count;//
+-----+
| count |
+-----+
|      1 |
+-----+
1 row in set (0.00 sec)

```

```

insert into emp values("Sneha","Latha",203,18,'F',987654,"Guntur",null,5);
select * from emps_count;

```

OUTPUT:-

```

mysql> select * from emps_count;//
+-----+
| count |
+-----+
|      2 |
+-----+
1 row in set (0.00 sec)

```

2) Create a trigger for backing up details of employees who are deleted from emp table.

```

create table backup_emps(Ename char(30),emp_id int,sal long,addr varchar(100),primary
key(emp_id));

```

```

create trigger back_up

```

```

before delete on emp

```

```

for each row

```

```

begin
    insert into backup_emps      values(old.Fname,old.ssn,old.salary,old.address);

end;

/

delete from emp where ssn=171;

select * from backup;

```

OUTPUT:-

```

mysql> select * from backup_emps ;//
+-----+-----+-----+-----+
| Ename | emp_id | sal  | addr |
+-----+-----+-----+-----+
| surya |    171 | NULL | NULL |
+-----+-----+-----+-----+
1 row in set (0.00 sec)

```

3) Create table student with the following attributes:

ID,Sname,sub1,sub2,sub3,sub4,sub5,total, percentile.

Now create trigger which automatically calculates total percentile when remaining attributes are given.

```

create table student (ID int,Sname char(20),sub1 int,sub2 int,sub3 int,sub4 int,sub5 int,total
int,percentile float);

```

```

create trigger total_percent

before insert on student

for each row

begin

    set New.total=New.s1+New.s2+New.s3+New.s4+New.s5,

    New.percent=((New.s1+New.s2+New.s3+New.s4+New.s5)/500)*100;

end;

/

insert into student values(1,'Zenobia',90,98,78,45,67,0,0);

insert into student values(2,'Alex',95,69,82,54,74,0,0);

```

```
select * from student;
```

OUTPUT:-

```
mysql> select * from student;
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| ID   | Sname | sub1 | sub2 | sub3 | sub4 | sub5 | total | percentile |
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
| 1    | Zenobia | 90 | 98 | 78 | 45 | 67 | 378 | 75.6 |
| 2    | Alex   | 95 | 69 | 82 | 54 | 74 | 374 | 74.8 |
+-----+-----+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)
```

Even though we inserted total,percentile as 0,0 the trigger has been fired and total,percentile are calculated based on remaining attributes.

LAB 11 - PL/SQL Programs(Until Loops)

16) To print Hello World

```
CODE:

DECLARE

message varchar2(20):= 'Hello, World!';

BEGIN

dbms_output.put_line(message);

END;

/

OUTPUT:

'Hello, World!'
```

26. To implement comments

```
CODE:

DECLARE

-- variable declaration

message varchar2(20):= 'Hello, World!';

BEGIN

/*

* PL/SQL executable statement(s)

*/

dbms_output.put_line(message);

END;

/

OUTPUT:

Hello, World!
```

27. Variables declaration

```
CODE:

DECLARE

num1 INTEGER;
```

```
num2 REAL;
num3 DOUBLE PRECISION;

BEGIN

null;

END;

/

OUTPUT:

PL/SQL procedure successfully completed.
```

28. User defined subtypes

CODE:

```
DECLARE

SUBTYPE name IS char(20);
SUBTYPE message IS varchar2(100);
salutation name; greetings message;

BEGIN

salutation := 'Reader ';
greetings := 'Welcome to the World of PL/SQL';
dbms_output.put_line('Hello ' || salutation || greetings);

END;

/

OUTPUT:

Hello Reader      Welcome to the World of PL/SQL
```

29. Initialization of variables and operations

CODE:

```
DECLARE

a integer := 10;
b integer := 20;
c integer;
f real;

BEGIN

c := a + b;

dbms_output.put_line('Value of c: ' || c);
```

[illegible]

30. **Local and Global variables in its simple form**

CODE:

```

DECLARE
-- Global variables
num1 number := 95;
num2 number := 85;

BEGIN
dbms_output.put_line('Outer Variable num1: ' || num1);
dbms_output.put_line('Outer Variable num2: ' || num2);

DECLARE
-- Local variables
num1 number := 195;
num2 number := 185;

BEGIN
dbms_output.put_line('Inner Variable num1: ' || num1);
dbms_output.put_line('Inner Variable num2: ' || num2);
END; dbms_output.put_line('Outer Variable num2: ' || num2);
END;

/

OUTPUT:
Outer Variable num1: 95
Outer Variable num2: 85
Inner Variable num1: 195
Inner Variable num2: 185
Outer Variable num2: 85

```

31. Assigning SQL Query Results to PL/SQL Variables

CODE:

```
CREATE TABLE CUSTOMERS(  
  ID INT NOT NULL,  
  NAME VARCHAR (20) NOT NULL,  
  AGE INT NOT NULL,  
  ADDRESS CHAR (25),  
  SALARY DECIMAL (18, 2),  
  PRIMARY KEY (ID)  
);  
  
INSERT INTO CUSTOMERS (ID,NAME,AGE,ADDRESS,SALARY)  
VALUES (1, 'Rayala', 32, 'IIIT', 2000.00 );  
  
INSERT INTO CUSTOMERS (ID,NAME,AGE,ADDRESS,SALARY)  
VALUES (2, 'Rayala Pavan', 25, 'TS', 1500.00 );  
  
INSERT INTO CUSTOMERS (ID,NAME,AGE,ADDRESS,SALARY)  
VALUES (3, 'Thanuj', 23, 'AP', 2000.00 );  
  
INSERT INTO CUSTOMERS (ID,NAME,AGE,ADDRESS,SALARY)  
VALUES (4, 'Chaitali', 25, 'Mumbai', 6500.00 );  
  
INSERT INTO CUSTOMERS (ID,NAME,AGE,ADDRESS,SALARY)  
VALUES (5, 'Kamala', 27, 'Hyd', 8500.00 );  
  
INSERT INTO CUSTOMERS (ID,NAME,AGE,ADDRESS,SALARY)  
VALUES (6, 'Komal', 22, 'MP', 4500.00 );  
  
/*SELECT INTO clause of SQL */  
  
DECLARE  
  
c_id customers.id%type :=&c_id;  
c_name customers.name%type;  
c_addr customers.address%type; c_sal customers.salary%type;  
  
BEGIN  
  
SELECT name, address, salary INTO c_name, c_addr, c_sal  
FROM customers  
  
WHERE id = c_id;  
  
dbms_output.put_line  
( 'Customer ' || c_name || ' from ' || c_addr || ' earns ' ||  
c_sal);
```

```
END;

/

OUTPUT:

Enter value for c_id: 2

old 2: c_id customers.id%type :=&c_id;
new 2: c_id customers.id%type :=2;

Customer Rayala Pavan from TS          earns 1500
```

32. Declaring a constant

CODE:

```
PI CONSTANT NUMBER := 3.141592654;

DECLARE

-- constant declaration

pi constant number := 3.141592654;

-- other declarations

radius number(5,2);

dia number(5,2);

circumference number(7, 2);

area number (10, 2);

BEGIN

-- processing

radius := 9.5;

dia := radius * 2;

circumference := 2.0 * pi * radius;

area := pi * radius * radius;

-- output

dbms_output.put_line('Radius: ' || radius);

dbms_output.put_line('Diameter: ' || dia);

dbms_output.put_line('Circumference: ' || circumference);

dbms_output.put_line('Area: ' || area);

END;

/

OUTPUT:

Radius: 9.5
```

Diameter: 19

Circumference: 59.69

Area: 283.53

33. The PL/SQL Literal ' printing

CODE:

```
DECLARE message varchar2(30):= 'That's power of DBMS!';
```

```
BEGIN
```

```
dbms_output.put_line(message);
```

```
END;
```

```
/
```

OUTPUT:

That's power of DBMS!

10)Operators

i. Arithmetic operators

CODE:

```
BEGIN
```

```
dbms_output.put_line( 10 + 5);
```

```
dbms_output.put_line( 10 - 5);
```

```
dbms_output.put_line( 10 * 5);
```

```
dbms_output.put_line( 10 / 5);
```

```
dbms_output.put_line( 10 ** 5);
```

```
END;
```

```
/
```

OUTPUT:

15

5

50

2

100000

ii. Relational operators

CODE:

DECLARE

a number (2) := &a;

b number (2) := &b;

BEGIN

IF (a = b) then

dbms_output.put_line('Line 1 - a is equal to b');

ELSE

dbms_output.put_line('Line 1 - a is not equal to b');

END IF;

IF (a < b) then

dbms_output.put_line('Line 2 - a is less than b');

ELSE

dbms_output.put_line('Line 2 - a is not less than b');

END IF;

IF (a > b) THEN

dbms_output.put_line('Line 3 - a is greater than b');

ELSE

dbms_output.put_line('Line 3 - a is not greater than b');

END IF; -- Lets change value of a and b

a := 5;

b := 20;

IF (a <= b) THEN

dbms_output.put_line('Line 4 - a is either equal or less
than b');

END IF;

IF (b >= a) THEN

dbms_output.put_line('Line 5 - b is either equal or greater
than a');

END IF;

IF (a <> b) THEN

dbms_output.put_line('Line 6 - a is not equal to b');

ELSE

dbms_output.put_line('Line 6 - a is equal to b');

END IF;

END;

/

OUTPUT:

Enter value for a: 20

old 2: a number (2) := &a;

new 2: a number (2) := 20;

Enter value for b: 12

old 3: b number (2) := &b;

new 3: b number (2) := 12;

Line 1 - a is not equal to b

Line 2 - a is not less than b

Line 3 - a is greater than b

Line 4 - a is either equal or less
than b

Line 5 - b is either equal or greater
than a

Line 6 - a is not equal to b

iii. Pattern matching operator

CODE:

DECLARE

PROCEDURE compare (value varchar2, pattern varchar2) is

BEGIN

IF value LIKE pattern THEN

dbms_output.put_line ('True');

ELSE

dbms_output.put_line ('False');

END IF;

END;

BEGIN

compare('Upendar', 'U%d_r');

compare('Upendar rao', 'Z%r_i');

END;


```
/
DECLARE
x number(2) := 10; BEGIN
IF (x between 5 and 20) THEN
dbms_output.put_line('True');
ELSE
dbms_output.put_line('False');
END IF;
IF (x BETWEEN 5 AND 10) THEN
dbms_output.put_line('True');
ELSE
dbms_output.put_line('False');
END IF;
IF (x BETWEEN 11 AND 20) THEN
dbms_output.put_line('True');
ELSE
dbms_output.put_line('False');
END IF;
END;
```

```
/
OUTPUT:
True
True
False
```

CODE:

```
DECLARE
letter varchar2(1) := 'm';
BEGIN
IF (letter in ('a', 'b', 'c')) THEN
dbms_output.put_line('True');
ELSE
dbms_output.put_line('False');
END IF;
```

```
IF (letter in ('m', 'n', 'o')) THEN
dbms_output.put_line('True');
ELSE
dbms_output.put_line('False');
END IF;
IF (letter is null) THEN
dbms_output.put_line('True'); ELSE
dbms_output.put_line('False');
END IF;
END;
/
OUTPUT:
False
True
False
```

iv. Logical operators

CODE:

```
DECLARE
a boolean := true;
b boolean := false;
BEGIN
IF (a AND b) THEN
dbms_output.put_line('Line 1 - Condition is true');
END IF;
IF (a OR b) THEN
dbms_output.put_line('Line 2 - Condition is true');
END IF;
IF (NOT a) THEN
dbms_output.put_line('Line 3 - a is not true');
ELSE
dbms_output.put_line('Line 3 - a is true');
END IF;
IF (NOT b) THEN
```

```
dbms_output.put_line('Line 4 - b is not true');
```

```
ELSE
```

```
dbms_output.put_line('Line 4 - b is true');
```

```
END IF;
```

```
END;
```

```
/
```

OUTPUT:

Line 2 - Condition is true

Line 3 - a is true

Line 4 - b is not true

v. Operator precedence

CODE:

```
DECLARE
```

```
a number(2) := 20;
```

```
b number(2) := 10;
```

```
c number(2) := 15;
```

```
d number(2) := 5;
```

```
e number(2) ;
```

```
BEGIN e := (a + b) * c / d; -- ( 30 * 15 ) / 5
```

```
dbms_output.put_line('Value of (a + b) * c / d is : ' || e );
```

```
e := ((a + b) * c) / d; -- (30 * 15 ) / 5
```

```
dbms_output.put_line('Value of ((a + b) * c) / d is : ' || e );
```

```
e := (a + b) * (c / d); -- (30) * (15/5)
```

```
dbms_output.put_line('Value of (a + b) * (c / d) is : ' || e );
```

```
e := a + (b * c) / d; -- 20 + (150/5)
```

```
dbms_output.put_line('Value of a + (b * c) / d is : ' || e );
```

```
END;
```

```
/
```

OUTPUT:

Value of (a + b) * c / d is : 90

Value of ((a + b) * c) / d is : 90

Value of (a + b) * (c / d) is : 90

Value of a + (b * c) / d is : 50

11)Conditional Statements

i. IF THEN

CODE:

DECLARE

a number(2) := 10;

BEGIN

a:= 10;

-- check the boolean condition using if statement

IF(a < 20) THEN

-- if condition is true then print the following

dbms_output.put_line('a is less than 20 ');

END IF;

dbms_output.put_line('value of a is : ' || a);

END;

/

OUTPUT:

a is less than 20

value of a is : 10

CODE:

DECLARE

c_id customers.id%type := &c_id;

c_sal customers.salary%type;

BEGIN

SELECT salary

INTO c_sal

FROM customers

WHERE id = c_id; IF (c_sal <= 2000) THEN

UPDATE customers

SET salary = salary + 1000

WHERE id = c_id;

dbms_output.put_line ('Salary updated');

```
END IF;

END;

/

OUTPUT:

Enter value for c_id: 3
old 2: c_id customers.id%type := &c_id;
new 2: c_id customers.id%type := 3;
Salary updated
```

ii. IF THEN ELSE

CODE:

```
DECLARE

a number(3) := 100;

BEGIN

-- check the boolean condition using if statement
IF( a < 20 ) THEN

-- if condition is true then print the following
dbms_output.put_line('a is less than 20 ');

ELSE

dbms_output.put_line('a is not less than 20 ');

END IF;

dbms_output.put_line('value of a is : ' || a);

END;

/

OUTPUT:

a is not less than 20
value of a is : 100
```

iii. IF THEN ELSE IF

CODE:

```
DECLARE

a number(3) := 100;

BEGIN

IF ( a = 10 ) THEN
```

```

dbms_output.put_line('Value of a is 10' );
ELSIF ( a = 20 ) THEN
dbms_output.put_line('Value of a is 20' );
ELSIF ( a = 30 ) THEN
dbms_output.put_line('Value of a is 30' );
ELSE
dbms_output.put_line('None of the values is matching');
END IF;
dbms_output.put_line('Exact value of a is: ' || a );
END;
/
OUTPUT:
None of the values is matching
Exact value of a is: 100

```

13)CASE Statement

CODE:

```

DECLARE
grade char(1) := &grade;
BEGIN
CASE grade
when 'A' then dbms_output.put_line('Excellent');
when 'B' then dbms_output.put_line('Very good');
when 'C' then dbms_output.put_line('Well done');
when 'D' then dbms_output.put_line('You passed');
when 'F' then dbms_output.put_line('Better try again');
else dbms_output.put_line('No such grade');
END CASE;
END;
/
OUTPUT:
Enter value for grade: 'A'
old 2: grade char(1) := &grade;
new 2: grade char(1) := 'A';

```

Excellent

14)Searched case

CODE:

DECLARE

grade char(1) := 'B';

BEGIN

case

when grade = 'A' then dbms_output.put_line('Excellent');

when grade = 'B' then dbms_output.put_line('Very good');

when grade = 'C' then dbms_output.put_line('Well done');

when grade = 'D' then dbms_output.put_line('You passed');

when grade = 'F' then dbms_output.put_line('Better try
again');

else dbms_output.put_line('No such grade');

end case;

END;

/

OUTPUT:

Very good

14)Nested IF then

CODE:

DECLARE

a number(3) := 100;

b number(3) := 200; BEGIN

-- check the boolean condition

IF(a = 100) THEN

-- if condition is true then check the following

IF(b = 200) THEN

-- if condition is true then print the following

dbms_output.put_line('Value of a is 100 and b is 200');

END IF;

END IF;

```
dbms_output.put_line('Exact value of a is : ' || a );
dbms_output.put_line('Exact value of b is : ' || b );

END;

/

OUTPUT:

Value of a is 100 and b is 200

Exact value of a is : 100

Exact value of b is : 200
```

15)LOOPS

➤ Basic loop

CODE:

```
DECLARE
x number := 10;
BEGIN
LOOP
dbms_output.put_line(x);
x := x + 10;
IF x > 50 THEN
exit;
END IF;
END LOOP;
-- after exit, control resumes here
dbms_output.put_line('After Exit x is: ' || x);
END;

/

OUTPUT:

10
20
30
40
50

After Exit x is: 60
```


ii. Loop breakup using exit when

CODE:

DECLARE

x number := 10;

BEGIN

LOOP dbms_output.put_line(x);

x := x + 10;

exit WHEN x > 50;

END LOOP;

-- after exit, control resumes here

dbms_output.put_line('After Exit x is: ' || x);

END;

/

OUTPUT:

10

20

30

40

50

After Exit x is: 60

iii. While loop

CODE:

DECLARE

a number(2) := 10;

BEGIN

WHILE a < 20 LOOP

dbms_output.put_line('value of a: ' || a);

a := a + 1;

END LOOP;

dbms_output.put_line('value of a: ' || a);

END;

/

OUTPUT:

value of a: 10

value of a: 11

value of a: 12

value of a: 13

value of a: 14

value of a: 15

value of a: 16

value of a: 17

value of a: 18

value of a: 19

value of a: 20

iv. For loop

CODE:

DECLARE

a number(2);

BEGIN

FOR a in 10 .. 20 LOOP

dbms_output.put_line('value of a: ' || a);

exit when a>15;

END LOOP;

END;

/

OUTPUT:

value of a: 10

value of a: 11

value of a: 12

value of a: 13

value of a: 14

value of a: 15

value of a: 16

v.Reverse for loop

CODE:

```
DECLARE
a number(2) ;
BEGIN FOR a IN REVERSE 10 .. 20 LOOP
dbms_output.put_line('value of a: ' || a);
exit WHEN a<15;
END LOOP;
END;
/
```

OUTPUT:

value of a: 20

value of a: 19

value of a: 18

value of a: 17

value of a: 16

value of a: 15

value of a: 14

vi. Nested for loops

CODE:

```
DECLARE
i number(3);
j number(3);
BEGIN
i := 2;
LOOP
j:= 2;
LOOP
exit WHEN ((mod(i, j) = 0) or (j = i));
j := j +1;
END LOOP;
IF (j = i ) THEN
dbms_output.put_line(i || ' is prime');
END IF;
i := i + 1;
exit WHEN i = 50;
```

```
END LOOP;
```

```
END;
```

```
/
```

```
OUTPUT:
```

```
2 is prime
```

```
3 is prime
```

```
5 is prime
```

```
7 is prime
```

```
11 is prime
```

```
13 is prime
```

```
17 is prime
```

```
19 is prime
```

```
23 is prime
```

```
29 is prime
```

```
31 is prime
```

```
37 is prime
```

```
41 is prime
```

```
43 is prime
```

```
47 is prime
```

vii. Exit

```
CODE:
```

```
DECLARE
```

```
a number(2) := 10;
```

```
BEGIN
```

```
-- while loop execution
```

```
WHILE a < 20 LOOP
```

```
dbms_output.put_line ('value of a: ' || a);
```

```
a := a + 1;
```

```
IF a > 15 THEN
```

```
    EXIT;
```

```
END IF;
```

```
END LOOP;
```

```
END;
```

```
/
```

OUTPUT:

value of a: 10

value of a: 11

value of a: 12

value of a: 13

value of a: 14

value of a: 15

viii. Goto

CODE:

DECLARE

a number(2) := 10;

BEGIN

<<loopstart>>

-- while loop execution

WHILE a < 20 LOOP

dbms_output.put_line ('value of a: ' || a);

a := a + 1;

IF a = 15 THEN

a := a + 1;

GOTO loopstart;

END IF;

END LOOP;

END;

/

OUTPUT:

value of a: 10

value of a: 11

value of a: 12

value of a: 13

value of a: 14

value of a: 16

value of a: 17

value of a: 18

value of a: 19

LAB-12 LOOPS AND PROCEDURES IN PL/SQL

Loops:

Basic loop:

Sequence of statements is enclosed between the loop and the END loop statements. Those statements are repeated until the given condition is true.

DECLARE

```
x number:= 10;
```

BEGIN

```
Loop
```

```
    dbms_output.put_line (x);
```

```
    IF X=50 THEN EXIT;
```

```
    END IF
```

```
    x:=X+10;
```

```
END Loop;
```

END;

Output:

10

20

30

40

50

WHILE Loop:

Repeats a statement (or) group of statements, while a given condition is true. It tests the Condition before executing the loop body.

Example:

DECLARE

```
a number(2) = 10;
```

BEGIN

```
while a<50 LOOP
```

```
    dbms_output.put_line(a);
```

```
    a:=a+1;
```

```
END loop;
```

```
dbms_output.put_line ('The value of a is: ' || a);
```

END;

/

Output:

10

20

30

40

The value of a is: 50

FOR LOOP:

Execute a sequence of statements multiple times and abbreviates the code that manages the loop.

Example:

DECLARE

a number(2);

BEGIN

FOR a IN 10..20 LOOP

dbms_output.put_line('Value of a is: ' || a);

EXIT WHEN a>15;

END LOOP;

END;

/

Output:

10

12

13

14

15

16

Arrays:

The PL/SQL language provides a data structure called the VARRAY, which can store a fixed size sequential collection of elements of the same type.

An array is a part of collection type data and it stands for variable size array. Each element in with it a varray has an index associated with it.

Syntax: CREATE OR REPLACE TYPE varray_type_name IS VARRAY(n) OF <element-types>, where Varray-type-name is a valid attribute name n, n is the number of elements in the array element type is the data type of the elements of the array.

DECLARE

type namesarray Is VARRAY(6) OF VARCHAR(10);

type grades IS VARRAY(6) OF INTEGER;

names namesarray;

marks grades;

total INTEGER;

BEGIN

FOR i IN 1..6 LOOP

names:=names array('Murari','Janav','Kesav','Indra','Charan','Harshith');

marks: = grades (98,97, 78, 87,72, 89);

total:= names .count;

dbms_output.put_line('Student: ' || names(i) || 'Marks: ' || marks(i));

END Loop;

END;

/

Output :

Total 6 students

Student: Murari Marks: 98

Student: Janav Marks: 97

Student: Kesava Marks: 78

Student: Indra Marks: 87

Student: Charan Marks: 72

Student: Harshith Marks: 89

Procedure:

It is a subprogram which does not return a Value directly, mainly used to perform an action.

Syntax:

```
CREATE [OR REPLACE] PROCEDURE procedure_name [(parameter_name [IN | OUT | IN OUT] type  
[, ...])] {IS | AS}
```

```
BEGIN
```

```
    < procedure_body >
```

```
END procedure_name;
```

Types of parameters in a procedure:-**IN:**

An IN parameter lets you pass a value to the subprogram. It is a read-only parameter. Inside the subprogram. An IN parameter acts like a constant. It cannot be assigned a value.

OUT:

An OUT parameter returns a value to the calling program. Inside the subprogram, an OUT parameter acts like a variable.

IN OUT:

An IN OUT parameter passes an initial value to a subprogram and returns an updated value to the caller.

Ex1:- (using IN and OUT parameters)

```
DECLARE
```

```
    a number;
```

```
    b number;
```

```
    c number;
```

```
PROCEDURE findMax(x IN number, y IN number, z OUT number) IS
```

```
BEGIN
```

```
    IF x > y THEN
```



```

        z:= x;
    ELSE
        z:= y;
    END IF;
END;
BEGIN
    a:= 23;
    b:= 45;
    findMax(a, b, c);
    dbms_output.put_line(' Maximum of (23, 45) : ' || c);
END;
/

```

Output:

Maximum of (23,45): 45

Ex (using IN OUT To find square of lype parameter) a number.

DECLARE

```

    a number;
PROCEDURE squareNum(x IN OUT number) IS
BEGIN
    x:= x * x;
END;
BEGIN
    a:= 23;
    squareNum(a);
    dbms_output.put_line(' Square of (23): ' || a);
END;
/

```

Output: Square of (23): 529

Functions:

These subprograms return à single value; mainly used to compute and return a value.

Syntax:-

```

CREATE (OR REPLACE] FUNCTION function name [parameter name (IN/OUT/IN OUT] type [...]]
return return-datatype {IS/AS} BEGIN function_body END function-name;

```

Ex:- Finding factorial of a number using recursion

DECLARE

```

    num number;
    factorial number;
FUNCTION fact(x number) RETURN number IS
    f number;
BEGIN

```

```

        IF x=0 THEN
            f := 1;
        ELSE
            f := x * fact(x-1);
        END IF;
    RETURN f;
END;
BEGIN
    num:= 6;
    factorial := fact(num);
    dbms_output.put_line(' Factorial ' || num || ' is ' || factorial);
END;
/

```

Output:- Factorial of 6 is 720.

Cursors:

A Cursor is a pointer to the context area PL/SQL Controls the Context area through a cursor. A Cursor holds the rows returned by a sql statement. The set of rows the cursor holds is referred to as the active set.

There are Two types of cursors.

1) Implicit Cursors: These are automatically created by the program whenever an sql statement is executed when there is no explicit cursor for the statement. These are not controlled by programmers.

The implicit cursor has attributes such %FOUND,%ISOPEN,%NOT FOUND,% ROW COUNT.

Let us consider a customers table.

mysql> SELECT * FROM CUSTOMERS;

```

+-----+-----+
| CNAME  | SALARY  |
+-----+-----+
| Maria   | 17000   |
| Catherine | 19000   |
| Mary    | 16000   |
| Isabella | 24000   |
| Leana   | 39000   |
| Anne    | 45000   |
| Zenobia | 50000   |
+-----+-----+

```

DECLARE

total_rows number;

BEGIN

```

    UPDATE customers
    SET salary = salary + 500;

```

```

        IF sql%notfound THEN
            dbms_output.put_line('no customers selected');
        ELSIF sql%found THEN
            total_rows := sql%rowcount;
            dbms_output.put_line( total_rows || ' customers selected ');
        END IF;
        IF sql%isopen then
            dbms_output.put_line( 'implicit cursor still open ');
        ELSE
            dbms_output.put_line('implicit cursor closed, not possible to control ');
        END IF;

    END;
/

```

2) Explicit cursors:

Explicit cursors are programmer defined Cursors for gaining more control over the context area. An explicit cursor should be defined in declaration section of PL/SQL block. It is created on a select Statement which returns more than one row.

Syntax: cursor_name IS select_statement;

example:

```

CURSOR c_customers IS
    SELECT id, name, address FROM customers :

```

/* Opening the Cursor

Opening the cursor allocates the memory for the cursor and makes it ready for fetching the rows returned by the SQL statement into it. For example, we will open the above defined cursor as follows –*/

Syntax: OPEN cursorname;

ex: OPEN c_customers;

/* Fetching the Cursor

Fetching the cursor involves accessing one row at a time. For example, we will fetch rows from the above-opened cursor as follows –

ex: FETCH c_customers INTO c_id, c_name, c_addr;

/* Closing the Cursor

Closing the cursor means releasing the allocated memory. For example, we will close the above opened cursor as follows –*/

ex: CLOSE c_customers;

EXAMPLE PROGRAM:**DECLARE**

c_id customers.id%type;

c_name customers.name%type;

c_addr customers.address%type;

CURSOR c_customers is

SELECT id, name, address FROM customers;

BEGIN

OPEN c_customers;

LOOP

FETCH c_customers into c_id, c_name, c_addr;

EXIT WHEN c_customers%notfound;

dbms_output.put_line(c_id || ' ' || c_name || ' ' || c_addr);

END LOOP;

IF c_customers%isopen then

dbms_output.put_line('explicit cursor still open ');

else dbms_output.put_line('implicit cursor closed, not possible to control ');

end if;

CLOSE c_customers;

IF c_customers%isopen then

dbms_output.put_line('explicit cursor still open ');

else dbms_output.put_line('explicit cursor is closed ');

end if;

END;

/

OUTPUT:

```
+-----+-----+-----+
| id | cname | address |
+-----+-----+-----+
| 1 | Maria | Austria |
| 2 | Catherine | Russia |
| 3 | Mary | Scotland |
| 4 | Isabella | Netherlands |
| 5 | Eleanor | Aquitaine |
| 6 | Anne | England |
| 7 | Zenobia | Egypt |
+-----+-----+-----+
```